基于 RV32I 指令集的 RISC-V 微处理器设计

实验报告

姓 名 吴溪语

学 号 3230103327

实验日期 2025.12.25

指导老师 屈民军、唐奕

Contents

1 实验目的 **1**

2 实验任务与要求 **1**

3 实验设备 **3**

3.1 RISC-V RV32I 指令集与基本执行机制 3

3.2 五级流水线处理器结构与工作流程 4

3.3 流水线冒险及其处理方法 6

3.3.1 数据冒险 6

3.3.2 控制冒险处理机制 7

3.3.3 冒险检测条件 7

4 实验过程 **7**

4.1 ID 模块的设计 7

4.1.1 指令译码（包含立即数产生电路）子模块的设计 8

4.1.2 寄存器堆 Registers 子模块的设计 14

4.1.3 分支检测 Branch Test 的设计 16

4.1.4 冒险检测功能电路 Hazard Detector 的设计 18

4.1.5 ID 顶层设计 19

4.2 EX 模块的设计 22

4.2.1 ALU 子模块的设计 22

4.2.2 EX 顶层及数据前推电路的设计 24

4.3 DataRAM 模块的设计 26

4.4 IF 模块的设计 28

4.5 流水线寄存器的设计 31

4.5.1 IF/ID 流水线寄存器 31

4.5.2 ID/EX 流水线寄存器 32

4.5.3 EX/MEM 流水线寄存器 33

4.5.4 MEM/WB 流水线寄存器 34

4.6 顶层文件的设计 35

5 仿真与结果分析 **39**

6 思考题 **45**

7 遇到的问题和解决方案 **46**

8 设计总结 **46**

1 实验目的

（1） 熟悉 RISC-V 指令系统。

（2） 了解提高 CPU 性能的方法。

（3） 掌握流水线 RISC-V 微处理器的工作原理。

（4） 理解数据冒险、控制冒险的概念以及流水线冲突的解决方法。

（5） 掌握流水线 RISC-V 微处理器的测试方法。

（6） 了解用软件实现数字系统的方法。

2 实验任务与要求

设计并实现一个基于 RV32I 指令集的五级流水线 RISC-V 微处理器，具体要求如下：

1. 至少能够正确运行下列 RV32I 核心指令：

   （1） 算术运算指令：add、sub、addi；

   （2） 逻辑运算指令：and、or、xor、slt、sltu、andi、ori、xori、slti、sltiu；

   （3） 移位指令：sll、srl、sra、slli、srli、srai；

   （4） 条件分支指令：beq、bne、blt、bge、bltu、bgeu；

   （5） 无条件跳转指令：jal、jalr；

   （6） 数据传送指令：lw、sw、lui、auipc；

   （7） 空指令：nop。

2. 采用五级流水线结构设计，并针对流水线中的数据冒险问题，实现数据前推或流水线阻塞等处理机制。

3. 在 Nexys Video 开发系统上实现所设计的 RISC-V 微处理器，要求 CPU 的运行主频不低于 25 MHz。

4. 实验内容：

   （1） 从网络下载相关实验文件；

   （2） 编写指令译码单元 Decode 模块的 Verilog HDL 代码，并使用 ModelSim 进行功能仿真；

   （3） 编写寄存器堆 Register 模块的 Verilog HDL 代码；

(4) 编写 ID 模块的 Verilog HDL 代码，ID.v 文件已给出端口列表；

(5) 编写 ALU 模块的 Verilog HDL 代码，并使用 ModelSim 进行功能仿真；

(6) 编写执行单元 EX 模块的 Verilog HDL 代码，EX.v 文件已给出端口列表；

(7) 编写 IF 模块的 Verilog HDL 代码，并使用 ModelSim 进行功能仿真；

(8) 打开 Vivado 文件夹下的 `Risc5CPU.xpr` 工程，生成符合 CPU 要求的数据存储器 IP 内核；

(9) 编写 CPU 顶层 Verilog HDL 代码，并使用 ModelSim 进行功能仿真。由于工程中包含 IP 内核，仿真时需加入相应仿真库，方法参考实验 3。根据表 1 对仿真结果进行验证，其中表中数据均为十六进制，“–” 表示该处数值无意义；

表 30.11 测试程序的运行结果

reset	clk	PC	Instruction (ID)	JumpFlag	Stall	ALU_A	ALU_B	ALUResult	MemDout (MEM)
1	1	0	0	0	0	0	0	0	-
	2	4	00003f37 (lui)	0	0	-	-	-	-
	3	8	02000fe7 (jalr)	2	0	-	3000	3000	-
	4	20	0	0	0	4	4	8	-
	5	24	00001c63 (bne)	0	0	--	-	-	-
	6	28	042f0293 (addi)	0	0	-	-	-	-
	7	2c	01f00333 (add)	0	0	3000	42	3042	-
	8	30	406283b3 (sub)	0	0	0	8	8	-
	9	34	0053ee33 (or)	0	0	3042	8	303a	-
0	10	38	fc000ae3 (beq)	1	0	303a	3042	307a	-
	11	8	0	0	0	-	-	-	-
	12	c	01c02623 (sw)	0	0	-	-	-	-
	13	10	00432e83 (lw)	0	0	0	c	c	-
	14	14	002e9293 (sll)	0	1	8	4	c	-
	15			0	0	-	-	-	307a
	16	18	00432e03 (lw)	0	0	307a	2	c1e8	-
	17	1c	00733e33 (sltu)	0	0	8	4	c	-
	18	20	0000fef (jal)	2	0	8	303a	1	307a
	19	1c	0	0	0	1c	4	20	-
	20	20	0000fef (jal)	2	0	-	-	-	-
	21	1c	0	0	0	1c	4	20	-

Figure 1: 测试程序的运行结果

（10） 再次打开 Vivado 文件夹下的 `Risc5CPU.xpr` 工程，添加流水线 CPU 设计的全部代码，完成综合、实现并下载至 Nexys Video 开发板；

（11） 连接带有 HDMI 接口的显示器进行测试。首先将 SW0 置为低电平，使 RISC-V CPU 工作在“单步”运行模式。复位后，每按一次上方按键，CPU 执行一步，记录显示器上的结果，并对照表 30.11 验证设计的正确性。

3 实验设备

本实验所使用的主要实验设备如下：

(1). Vivado 2022.2 开发环境；

(2). Nexys Video 开发板；

(3). 具有 HDMI 接口的显示器。

3.1 RISC-V RV32I 指令集与基本执行机制

RISC-V RV32I 指令集是 RISC-V 架构的标准整数指令集，具有良好的扩展性与定制化特性，其基础整数指令集固定，可通过扩展模块实现复杂功能，是本实验微处理器设计的指令基础。

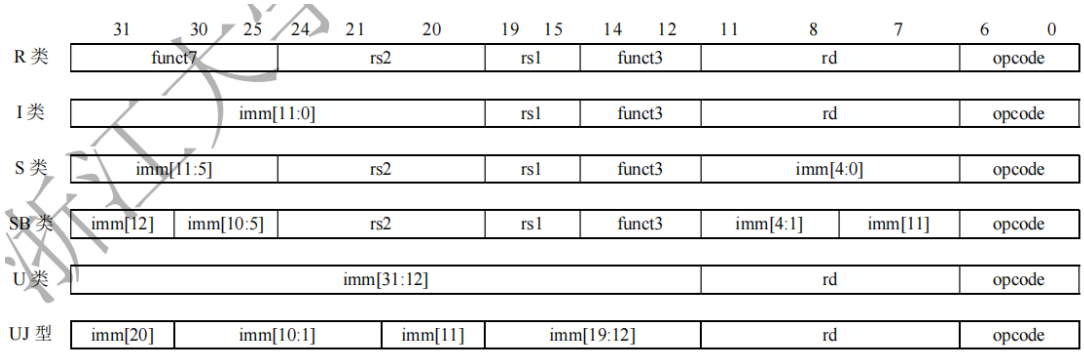


Figure 2: RV32I 指令格式

该指令集采用 32 位固定长度指令格式，定义了 R 型、I 型、S 型、SB 型、U 型、UJ 型六种核心指令格式。不同格式指令的字段（操作码 opcode、源寄存器号 rs1/rs2、目标寄存器号 rd、立即数 imm、功能扩展码 funct3/funct7 等）分布固定，仅立即数字段的存放位置随指令类型有所差异。

RV32I 指令集对应的寄存器堆为 32 位架构，包含 32 个 32 比特寄存器（编号为 0 ~ 31）。其中，第 0 号寄存器（X0）通过硬件连线固定为常数 0，其余寄存器均有明确功能定义，例如 X1 为返回地址寄存器（ra）、X2 为堆栈指针寄存器（sp）、X10 ~ X17 为函数参数与返回值寄存器（a0 ~ a7）等。

表 30.1 寄存器堆使用规范

寄存器编号	助记符	用途
X0	zero	硬件连线 0
X1	ra	Return address 返回地址
X2	sp	堆栈指针
X3	gp	全局指针
X4	tp	Thread（线程）指针
X5～X7	t0～t2	临时变量
X8	s0/fp	临时变量/帧指针
X9	s1	临时变量，过程调用时不需要恢复
X10～X17	a0~a7	函数参数，其中 a0、a1 用于存放函数返回值
X18～X27	s2~s11	保留
X28～X31	t3～t6	临时变量

Figure 3: 寄存器堆使用规范

RV32I 指令的基本执行遵循“取指—译码—执行—访存—回写”的五级处理流程。指令类型涵盖算术运算（add、sub、addi 等）、逻辑运算（and、or、xor 等）、移位运算（sll、srl、sra 等）、条件分支（beq、bne、blt 等）、无条件跳转（jal、jalr）、数据传送（lw、sw、lui、auipc）以及空指令（nop）。在指令执行过程中，通过解析操作码与功能扩展码确定运算类型，依据寄存器号读取操作数，完成运算后将结果回写至目标寄存器或写入存储器。

3.2 五级流水线处理器结构与工作流程

为提升微处理器的运行速度与吞吐量，本实验采用五级流水线技术，将指令执行过程拆解为取指令（IF）、指令译码（ID）、执行（EX）、存储器访问（MEM）和寄存器回写（WB）五个独立阶段。每个阶段在一个时钟周期内完成特定功能，不同指令的不同阶段可并行执行，从而实现流水作业。

五级流水线处理器的核心结构包括 IF、ID、EX、MEM 四大功能模块，以及 IF/ID、ID/EX、EX/MEM、MEM/WB 四组流水线寄存器。

流水线寄存器在时钟上升沿完成阶段间数据与控制信号的传递，实现各阶段的隔离与同步。其中，IF/ID 寄存器具备同步清零与保持功能，用于应对流水线阻塞与清空；ID/EX 寄存器具备同步清零功能，用于插入流水线气泡；EX/MEM 与 MEM/WB 为普通 D 型寄存器。

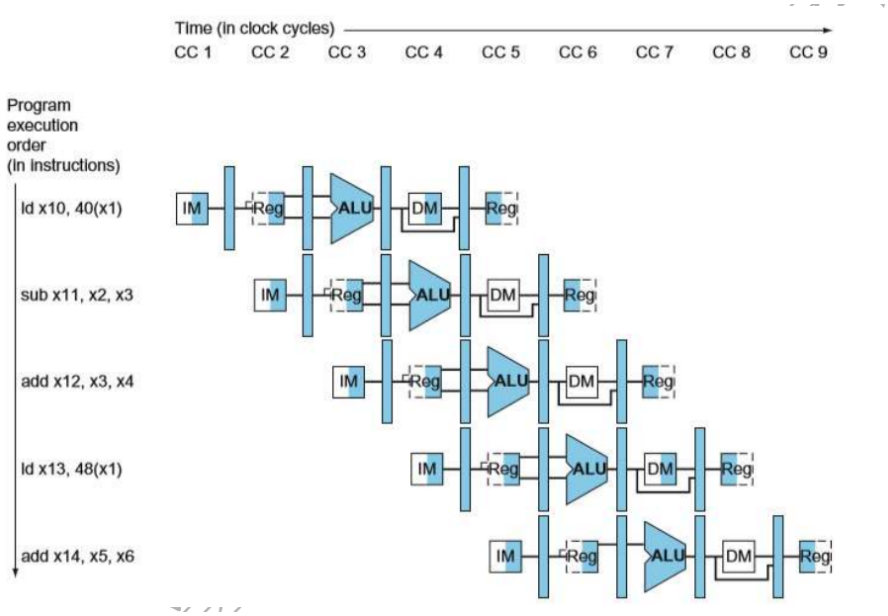


Figure 4: 五级流水线流水作业示意图

各阶段的具体工作流程如下：

1. **取指令阶段（IF）**：由 PC 寄存器、指令存储器（ROM）、加法器及选择器组成。根据 PC 指针从 ROM 中读取指令，同时计算下一条顺序执行指令的 PC 值（PC+4），并在时钟沿将指令与当前 PC 值送入 IF/ID 寄存器。核心控制信号包括 PCSource、IFWrite 和 IF_flush。

2. **指令译码阶段（ID）**：对 IF 阶段传入的指令进行译码，解析指令类型与寄存器号；通过寄存器堆读取源寄存器数据；生成后续阶段所需的控制信号；同时完成冒险检测与分支条件判断，若检测到冒险，则生成 Stall 信号阻塞流水线并插入气泡。

3. **执行阶段（EX）**：由算术逻辑单元（ALU）、数据前推电路及多路选择器组成。根据控制信号选择 ALU 操作数，完成加减、逻辑、移位和比较等运算；对于访存指令，在该阶段计算存储器访问地址。数据前推电路通过 ForwardA 和 ForwardB 信号实现数据转发。

4. **存储器访问阶段（MEM）**：仅对数据传送指令执行存储器读写操作。对于 sw 指令，将数据写入 RAM；对于 lw 指令，从 RAM 中读取数据；其他指令在该阶段仅完成数据传递。

5. **回写阶段（WB）**：根据 MemtoReg 信号选择回写数据来源，通过 RegWrite 信号控制写使能，将指令执行结果回写至寄存器堆。

3.3 流水线冒险及其处理方法

流水线冒险是指由于指令间存在数据依赖或控制流变化而导致的执行冲突问题，主要包括数据冒险和控制冒险。本实验以数据冒险处理为重点，同时针对控制冒险设计相应处理机制。

3.3.1 数据冒险

数据冒险源于指令之间的数据相关，即前一条指令的结果尚未写回寄存器，而后一条指令已需要读取该寄存器数据。根据相关程度不同，可分为一阶、二阶和三阶数据相关。

1. **一阶数据相关（EX 冒险）**：当前指令的 EX 阶段与前一条指令的 MEM 阶段并行，通过将前一条指令的 ALU 运算结果直接转发至当前指令的 EX 阶段解决。
2. **二阶数据相关（MEM 冒险）**：当前指令的 EX 阶段与前两条指令的 WB 阶段并行，通过转发回写数据至 EX 阶段解决。
3. **三阶数据相关**：通过寄存器堆支持同周期写后读（RAW），保证读取到最新写入的数据，无需额外处理。
4. **流水线阻塞与气泡插入**：对于 lw 指令后紧跟使用其结果的指令，采用流水线阻塞方式插入一个空操作，使数据相关延迟一个周期后再通过转发解决。

Figure 5: 一阶前推网络示意图

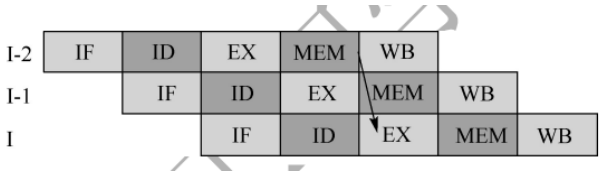


Figure 6: 二阶前推网络示意图

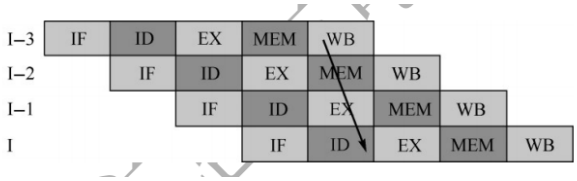


Figure 7: 三阶前推网络示意图

3.3.2 控制冒险处理机制

控制冒险由分支指令或跳转指令引起。本实验在 ID 阶段提前完成分支条件判断，若判断为跳转或分支成立，则通过 IF_flush 信号清空 IF/ID 寄存器，并将目标地址送入 PC，从而减少错误指令对流水线性能的影响。

3.3.3 冒险检测条件

数据冒险检测在 ID 阶段完成，当上一条指令为 lw 指令且其目标寄存器与当前指令源寄存器相同时，触发流水线阻塞。数据转发判断条件为前序指令写使能有效、目标寄存器非 X0 且与当前指令源寄存器相同。

4 实验过程

4.1 ID 模块的设计

指令译码模块（ID，Instruction Decode）的主要作用是对来自 IF 阶段的指令进行译码，并根据译码结果产生相应的控制信号，同时完成寄存器堆读取、分支条件判断以及部分流水线冒险检测工作。整个 CPU 中的大部分控制信号均在 ID 阶段生成，因此该模块在流水线处理器中具有核心地位。

本实验中设计的 ID 模块主要由指令译码（Decode）子模块、寄存器堆（Registers）子模块以及分支检测（Branch Test）子模块组成。各子模块在 ID 顶层模块中进行统一连接与调度，共同完成译码阶段的全部功能。ID 模块的输入输出接口如图所示。

表 30.3 ID 模块的输入/输出引脚说明

引脚名称	方向	说明
clk	Input	系统时钟
Instruction_id[31:0]		指令机器码
PC_id[31:0]		指令指针
RegWrite_wb		寄存器写允许信号，高电平有效
rdAddr_wb[4:0]		寄存器的写地址。
RegWriteData_wb[31:0]		写入寄存器的数据
MemRead_ex		冒险检测的输入
rdAddr_ex[4:0]		
MemtoReg_id	Output	决定回写的数据来源（0：ALU；1：存储器）
RegWrite_id		寄存器写允许信号，高电平有效
MemWrite_id		存储器写允许信号，高电平有效
MemRead_id		存储器读允许信号，高电平有效
ALUCode_id[3:0]		决定 ALU 采用何种运算
ALUSrcA_id		决定 ALU 的 A 操作数的来源（0：rs1；1：pc）
ALUSrcB_id[1:0]		决定 ALU 的 B 操作数的来源(2'b00：rs2；2'b01：imm；2'b10: 常数 4)
Stall		ID/EX 寄存器清空信号，高电平表示插入一个流水线气泡
Branch		条件分支指令的判断结果，高电平有效
Jump		无条件分支指令的判断结果，高电平有效
IFWrite		阻塞流水线的信号，低电平有效
BranchAddr[31:0]		分支地址
Imm_id[31:0]		立即数
rdAddr_id[4:0]		回写寄存器地址
rs1Addr_id[4:0]		两个数据寄存器地址
rs2Addr_id[4:0]		
rs1Data_id[31:0]		寄存器两个端口输出数据
rs2Data_id[31:0]		

Figure 8: ID 模块的接口信息

4.1.1 指令译码（包含立即数产生电路）子模块的设计

指令译码子模块的主要功能是根据输入的 32 位指令机器码判断指令类型，并产生对应的控制信号，同时生成立即数和分支偏移量。该模块采用纯组合逻辑实现。

根据 RV32I 指令集的规定，指令可分为 R 型、I 型、S 型、SB 型、U 型和 UJ 型。根据操作数的来源和立即数构成方式不同，再次细分指令。

- **R type 类指令：** 操作码（opcode，简称 op）为 7'h33。R 类指令的两个操作数分别来自寄存器 `rs1` 和 `rs2`。
- **I type 类指令：** 操作码为 7'h13。I 类算术逻辑运算指令及移位指令的两个操作数分别为寄存器 `rs1` 和立即数 `imm`。
- **LW 指令：** 操作码为 7'h03，属于 I 类的数据传送指令 `lw`，其两个操作数分别为寄存器 `rs1` 和立即数 `imm`。
- **JALR 指令：** 操作码为 7'h67，属于 I 类的无条件跳转指令 `jalr`，其两个操作数分别为程序计数器 `PC` 和常数 4。
- **SW 指令：** 操作码为 7'h23，属于 S 类的数据传送指令 `sw`，其两个操作数分别为寄存器 `rs1` 和立即数 `imm`。

- **SB type 类指令：** 操作码为 7'h63，SB 类条件分支指令的两个操作数分别为程序计数器 PC 和立即数 imm。
- **LUI 指令：** 操作码为 7'h37，属于 U 类的数据传送指令 lui，仅使用一个操作数，即立即数 imm。
- **AUIPC 指令：** 操作码为 7'h17，属于 U 类的数据传送指令 auipc，其两个操作数分别为程序计数器 PC 和立即数 imm。
- **JAL 指令：** 操作码为 7'h6F，属于 UJ 类的无条件跳转指令 jal，其两个操作数分别为程序计数器 PC 和常数 4。

各变量的值由式 30.3 决定。

$$\begin{cases} \text{R_type} = (\text{op} == \text{R_type_op}) \\ \text{I_type} = (\text{op} == \text{I_type_op}) \\ \text{SB_type} = (\text{op} == \text{SB_type_op}) \\ \text{LW} = (\text{op} == \text{LW_op}) \\ \text{JALR} = (\text{op} == \text{JALR_op}) \\ \text{SW} = (\text{op} == \text{SW_op}) \\ \text{LUI} = (\text{op} == \text{LUI_op}) \\ \text{AUIPC} = (\text{op} == \text{AUIPC_op}) \\ \text{JAL} = (\text{op} == \text{JAL_op}) \end{cases} \tag{30.3}$$

根据表达式，可得对应代码：

```
// 指令类型判别
wire R_type, I_type, SB_type, LW, JALR, SW, LUI, AUIPC, JAL;

assign R_type = (op == R_type_op);
assign I_type = (op == I_type_op);
assign SB_type = (op == SB_type_op);
assign LW     = (op == LW_op);
assign JALR   = (op == JALR_op);
assign SW     = (op == SW_op);
assign LUI    = (op == LUI_op);
assign AUIPC  = (op == AUIPC_op);
assign JAL    = (op == JAL_op);
```

(1). 只有 LW 指令需要从数据存储器中读取数据，且其回写数据来源于存储器，因此有

$$\text{MemtoReg} = LW$$

$$\text{MemRead} = LW$$

(2). 只有 SW 指令会向数据存储器写入数据，因此有

$$\text{MemWrite} = SW$$

(3). 需要进行寄存器写回的指令类型包括 R_type、I_type、LW、JALR、LUI、AUIPC 以及 JAL，因此寄存器写使能信号为

$$\text{RegWrite} = R_type \,||\, I_type \,||\, LW \,||\, JALR \,||\, LUI \,||\, AUIPC \,||\, JAL$$

(4). 无条件跳转指令仅包括 JALR 和 JAL，因此跳转控制信号为

$$\text{Jump} = JALR \,||\, JAL$$

上述对应代码为：

```
 // LW: 从存储器读数据并回写寄存器
assign MemtoReg=LW;
assign MemRead=LW;

// SW: 对存储器写数据
assign MemWrite=SW;

assign RegWrite =R_type || I_type || LW || JALR || LUI || AUIPC || JAL;

// 无条件跳转
assign Jump=JALR || JAL;
```

(5). 操作数 A 和操作数 B 的选择信号由各类指令的运算需求决定。通过分析不同指令对操作数来源的要求，可得到表 30.4 所示的操作数选择功能表。

表 30.4 操作数选择信号的功能表

类型	ALUSrcA_id	ALUSrcB_id[1:0]	说明
R_type	0	2'b00	rd=rs1 op rs2
I_type	0	2'b 01	rd=rs1 op imm
LW	0	2'b 01	rs1 + imm
SW	0	2'b 01	rs1 + imm
JALR	1	2'b 10	rd=pc + 4
JAL	1	2'b 10	rd=pc + 4
LUI	1'bx	2'b 01	rd= imm
AUIPC	1	2'b 01	rd=pc + imm

Figure 9: 操作数选择信号的功能表

由该表可进一步推导出操作数选择信号的逻辑表达式：

$$\text{ALUSrcA} = JALR \,||\, JAL \,||\, AUIPC$$

$$\text{ALUSrcB}[1] = JAL \mid\mid JALR$$

$$\text{ALUSrcB}[0] = \sim (R_type \mid\mid JAL \mid\mid JALR)$$

```
// ALU 操作数选择
assign ALUSrcA = JALR || JAL || AUIPC;
assign ALUSrcB[1] = JAL ||JALR;
assign ALUSrcB[0] = ~(R_type || JAL || JALR);
```

(6). **ALUCode 的确定**。除条件分支指令外，其余指令均需要 ALU 参与运算。根据 RV32I 指令集的要求，ALU 共需支持 11 种不同的运算类型，因此 ALUCode 信号采用 4 位二进制编码表示。ALUCode 的具体编码关系如表 30.5 所示。

需要注意的是，表中所示的 `funct7[6]` 与 `funct6[5]` 在指令中对应同一位，即 `instruction[30]`。

表 30.5 ALUCode 的功能表

R_type	I_type	LUI	funct3	funct7[6] (funct6[5])	ALUCode	备注
1	0	0	3'o0	0	4'd 0	加
1	0	0	3'o0	1	4'd 1	减
1	0	0	3'o1	0	4'd 6	左移 A<<B
1	0	0	3'o2	0	4'd9	A<B?1:0
1	0	0	3'o3	0	4'd10	A<B?1:0（无符号数）
1	0	0	3'o4	0	4'd 4	异或
1	0	0	3'o5	0	4'd 7	右移 A>>B
1	0	0	3'o5	1	4'd 8	算术右移 A>>>B
1	0	0	3'o6	0	4'd 5	或
1	0	0	3'o7	0	4'd 3	与
0	1	0	3'o0	x	4'd 0	加
0	1	0	3'o1	x	4'd 6	左移
0	1	0	3'o2	x	4'd9	A<B?1:0
0	1	0	3'o3	x	4'd10	A<B?1:0（无符号数）
0	1	0	3'o4	x	4'd 4	异或
0	1	0	3'o5	0	4'd 7	右移 A>>B
0	1	0	3'o5	1	4'd 8	算术右移 A>>>B
0	1	0	3'o6	x	4'd 5	或
0	1	0	3'o7	x	4'd 3	与
0	0	1	x	x	4'd 2	送数:ALUResult=B
其它					4'd 0	加

Figure 10: ALUcode 功能表

根据功能表，可以写出对应的 ALUcode 确定部分的代码：

```
// ALUCode 的确定
  always@(*) begin
      if (LUI&(~R_type)&(~I_type))
    ALUCode = alu_lui;
  else if((R_type|I_type)&(~LUI))
  begin
    case (funct3)
      ADD_funct3: begin
                if((R_type&(~funct6_7))|I_type)
```

```
                    ALUCode = alu_add;
                  else if(R_type&funct6_7)
                    ALUCode = alu_sub;
                  end
        SLL_funct3: begin
                  if((R_type&(~funct6_7))|I_type)
                    ALUCode = alu_sll;
                  end
        SLT_funct3: begin
                  if((R_type&(~funct6_7))|I_type)
                    ALUCode = alu_slt;
                  end
        SLTU_funct3:begin
                  if((R_type&(~funct6_7))|I_type)
                    ALUCode = alu_sltu;
                  end
        XOR_funct3: begin
                  if((R_type&(~funct6_7))|I_type)
                    ALUCode = alu_xor;
                  end
        SRL_funct3: begin
                  if((R_type&(~funct6_7))|(I_type&(~funct6_7)))
                    ALUCode = alu_srl;
                  else if((R_type&funct6_7)|(I_type&funct6_7))
                    ALUCode = alu_sra;
                  end
        OR_funct3: begin
                  if((R_type&(~funct6_7))|I_type)
                    ALUCode = alu_or;
                  end
        AND_funct3: begin
                  if((R_type&(~funct6_7))|I_type)
                    ALUCode = alu_and;
                  end

        default:  ALUCode = alu_add;
      endcase
    end
    else
      ALUCode = alu_add;
    end
```

(7). **立即数产生电路 (ImmGen) 设计**。I_type、SB_type、LW、JALR、SW、LUI、AUIPC 以及 JAL 等指令均需要使用立即数。由于 I 类指令中算术逻辑运算与移位运算的立即数构成方式不同，因此引入变量 Shift 对两类指令进行区分。当 Shift=1 时

表示移位运算指令，否则表示算术逻辑运算指令，其判断条件为

$$\text{Shift} = (funct3 == 1) \,||\, (funct3 == 5)$$

不同类型指令的立即数拼接及符号扩展方式如表 30.6 所示，其中表中的 `inst` 表示指令字 `instruction`。

表 30.6（a）立即数 imm 产生方法

类别	Shift	Imm	说明
I_type	1	{27'd0,inst[24:20]}	无符号数 0 扩展
I_type	0	{{20{inst[31]}},inst[31:20]}	有符号数符号扩展
LW	x		
SW	x	{{20{inst[31]}},inst[31:25],inst[11:7]}	
LUI	x	{inst[31:12], 12'd0}	
AUIPC	x		
other	x	-	

表 30.6（b）立即数 offset 产生方法

类别	offset	说明
JALR	{{20{inst[31]}},inst[31:20]}	有符号数符号扩展
JAL	{{11{inst[31]}}, inst[31], inst[19:12] , inst[20], inst[30:21], 1'b0}	
SB_type	{{19{inst[31]}}, inst[31], inst[7],inst[30:25] , inst[11:8], 1'b0}	
other	-	

Figure 11: 立即数产生方法

对应的代码设计如下：

```
always@(*) begin
  if(I_type) begin
    Imm = Shift?{26'd0, Instruction[25:20]}:{{20{Instruction[31]}}, Instruction
        [31:20]};
    offset = 32'bx;
  end
  else if(LW)begin
    Imm = {{20{Instruction[31]}}, Instruction[31:20]};
    offset = 32'bx;
  end
  else if(JALR) begin
    Imm = 32'bx;
    offset = {{20{Instruction[31]}}, Instruction[31:20]};
  end
  else if(SW) begin
    Imm = {{20{Instruction[31]}}, Instruction[31:25], Instruction[11:7]};
    offset= 32'bx;
  end
  else if(JAL) begin
    Imm = 32'bx;
    offset = {{11{Instruction[31]}}, Instruction[31], Instruction[19:12], Instruction
        [20], Instruction[30:21], 1'b0};
  end
  else if(LUI || AUIPC) begin
```

```
      Imm = {Instruction[31:12], 12'd0};
      offset= 32'bx;
    end
    else if(SB_type) begin
      Imm = 32'bx;
      offset = {{19{Instruction[31]}}, Instruction[31], Instruction[7], Instruction
          [30:25], Instruction[11:8], 1'b0};
    end
    else begin
      Imm = 32'bx;
      offset = 32'bx;
    end
  end
```

4.1.2 寄存器堆 Registers 子模块的设计

寄存器堆子模块用于实现 RV32I 架构中定义的通用寄存器文件。寄存器堆共包含 32 个 32 位寄存器，通过寄存器编号对寄存器进行访问，其中 0 号寄存器被规定为常数 0，在任何情况下均不允许被写入。

寄存器堆提供两个独立的读端口和一个写端口。读操作为组合逻辑，根据指令中给出的源寄存器地址直接输出对应寄存器的内容；写操作在时钟上升沿进行，由写回阶段传递而来的寄存器写使能信号控制。当写使能有效且目标寄存器编号不为 0 时，写回数据被写入对应寄存器。

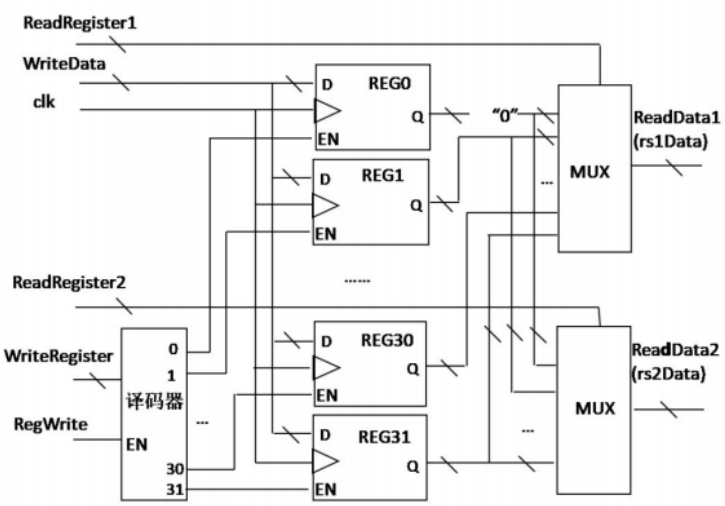


Figure 12: 寄存器堆原理图

对应的寄存器堆代码设计如下：

```
module RBWRegisters (clk, ReadRegister1, ReadRegister2, WriteRegister, WriteData,
    RegWrite, ReadData1, ReadData2);
```

```
    input clk;
    input[4:0] ReadRegister1, ReadRegister2, WriteRegister;
    input[31:0] WriteData;
    input RegWrite;
    output[31:0] ReadData1, ReadData2;

    reg[31:0] regs[31:0];
    assign ReadData1 = (ReadRegister1 == 5'b0)?32'b0:regs[ReadRegister1];
    assign ReadData2 = (ReadRegister2 == 5'b0)?32'b0:regs[ReadRegister2];
    always @(posedge clk) if(RegWrite) regs[WriteRegister] <= WriteData;
endmodule
```

在流水线型 CPU 设计中，寄存器堆设计还应解决三阶数据相关的数据转发问题。当满足三阶数据相关条件时，寄存器具有 Read After Write 特性。设计时，只需要在图 12 设计寄存器堆的基础上添加少量电路就可实现 Read After Write 特性，如下图所示。

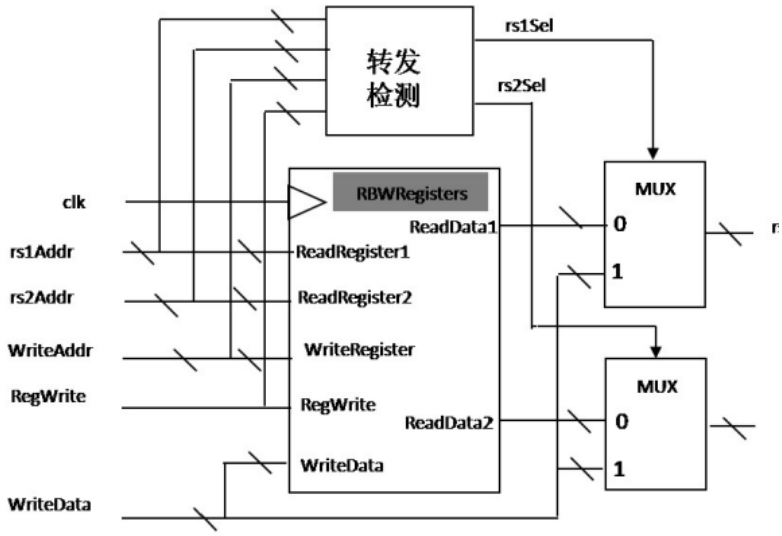


Figure 13: 具有 Read After Write 特性寄存器堆的原理框图

图中转发检测电路的输出表达式为

$$\text{rs1Sel} = \text{RegWrite} \;\&\&\; (\text{WriteAddr}! = 0) \;\&\&\; (\text{WriteAddr} == \text{rs1Addr})$$

$$\text{rs2Sel} = \text{RegWrite} \;\&\&\; (\text{WriteAddr}! = 0) \;\&\&\; (\text{WriteAddr} == \text{rs2Addr})$$

此时寄存器堆模块代码设计为:

```
module registers (clk, rs1Addr, rs2Addr, RegWrite, WriteAddr, WriteData, rs1Data,
    rs2Data);
    input clk, RegWrite;
    input[4:0] rs1Addr, rs2Addr, WriteAddr;
    input[31:0] WriteData;
    output[31:0] rs1Data, rs2Data;
```

```

    wire[31:0] ReadData1, ReadData2;
    RBWRegisters RBWRegisters0(
        // Inputs
        .clk(clk),
        .ReadRegister1(rs1Addr),
        .ReadRegister2(rs2Addr),
        .WriteRegister(WriteAddr),
        .WriteData(WriteData),
        .RegWrite(RegWrite),
        // Outputs
        .ReadData1(ReadData1),
        .ReadData2(ReadData2)
    );

    wire rs1Sel, rs2Sel;
    assign rs1Sel = RegWrite && (WriteAddr!=0) && (WriteAddr==rs1Addr);
    assign rs2Sel = RegWrite && (WriteAddr!=0) && (WriteAddr==rs2Addr);

    assign rs1Data = rs1Sel?WriteData:ReadData1;
    assign rs2Data = rs2Sel?WriteData:ReadData2;
endmodule
```

4.1.3 分支检测 Branch Test 的设计

分支检测电路主要用于判断分支条件是否成立，在 Verilog HDL 可以用比较运算符号“>”、“==”和“<”描述，但要注意符号数和无符号数的处理方法不同。在这里，我们用加法器来实现。

(1). 用一个 32 位加法器完成 rs1Data+（rs2Data）+1（即 rs1Data-rs2Data），设结果为 sum [31:0]。

(2). 确定比较运算的结果。对于比较运算来说，如果最高位不同，即 rs1Data [31] rs2Data [31]，可根据 rs1Data [31]、rs2Data [31] 决定比较结果，但是应注意符号数、无符号数的最高位 rs1Data [31]、rs2Data [31] 代表意义不同。若两数最高位相同，则两数之差不会溢出，所以比较运算结果可由两个操作数之差的符号位 sum[31] 决定。

在符号数比较运算中，rs1Data<rs2Data 有以下两种情况：

(a) rs1Data 为负数、rs2Data 为 0 或正数：rs1Data[31] && (~rs2Data[31])

(b) rs1Data、rs2Data 符号相同，sum 为负：(rs1Data[31] ~^rs2Data[31]) && sum[31]

因此，符号数 rs1Data<rs2Data 比较运算结果为

$$\text{isLT} = \text{rs1Data[31]\&\&(\sim rs2Data[31]) || (rs1Data[31]\sim\hat{}\text{rs2Data[31])\&\&sum[31]}}$$

同样地，无符号数比较运算中，rs1Data<rs2Data 有以下两种情况：

(a) rs1Data 最高位为 0、rs2Data 最高位为 1：(~rs1Data[31]) && rs2Data[31]

(b) rs1Data、rs2Data 最高位相同，sum 为负：(rs1Data[31] ~^rs2Data[31]) && sum[31]

因此，无符号数比较运算结果为

$$\text{isLTU} = \text{(\sim rs1Data[31])\&\&rs2Data[31] || (rs1Data[31]\sim\hat{}rs2Data[31])\&\&sum[31]}$$

最后用数据选择器完成下式即可完成分支检测。

$$\text{Branch} = \begin{cases} \sim(|\text{sum}[31:0]); & \text{SB_type} \ \&\&\ (\text{funct3} == \text{beq_funct3}) \\ |\text{sum}[31:0]; & \text{SB_type} \ \&\&\ (\text{funct3} = \text{bne_funct3}) \\ \text{isLT}; & \text{SB_type} \ \&\&(\text{funct3} = \text{blt_funct3}) \\ \sim \text{isLT}; & \text{SB_type} \ \&\&\ (\text{funct3} = \text{bge_funct3}) \\ \text{isLTU}; & \text{SB_type} \ \&\&\ (\text{funct3} = \text{bltu_funct 3}) \\ \sim \text{is}LTU; & \text{SB_type} \ \&\&\ (\text{funct3} = \text{bgeu_funct 3}) \\ 0 & \text{others} \end{cases}$$

分支检测模块代码设计如下：

```
module BranchTest(
    input[31:0] Instruction,
    input[31:0] rs1Data,
    input[31:0] rs2Data,
    output reg Branch
    );

    parameter SB_type_op =  7'b1100011;
    parameter BEQ_FUNCT3 =  3'o0;
    parameter BNE_FUNCT3 =  3'o1;
    parameter BLT_FUNCT3 =  3'o4;
    parameter BGE_FUNCT3 =  3'o5;
    parameter BLTU_FUNCT3 = 3'o6;
    parameter BGEU_FUNCT3 = 3'o7;

    wire[31:0] sum;
    Add_32bit adder_BT(
        .a(rs1Data),
```

```
        .b(~rs2Data),
        .ci(1),
        .sum(sum),
        .c_out()
    );

    wire isLT, isLTU;
    assign isLT = rs1Data[31]&&(~rs2Data[31]) || (rs1Data[31]~^rs2Data[31])&&sum[31];
    assign isLTU = (~rs1Data[31])&&rs2Data[31] || (rs1Data[31]~^rs2Data[31])&&sum[31];

    wire[6:0] op;
    wire[2:0] funct3;
    wire SB_type;

    assign op = Instruction[6:0];
    assign funct3 = Instruction[14:12];
    assign SB_type = (op == SB_type_op);

    always@(*)begin

    if(SB_type) begin
        case (funct3)
            BEQ_FUNCT3: Branch = ~(|sum[31:0]);
            BNE_FUNCT3: Branch = |sum[31:0];
            BLT_FUNCT3: Branch = isLT;
            BGE_FUNCT3: Branch = ~isLT;
            BLTU_FUNCT3:Branch = isLTU;
            BGEU_FUNCT3:Branch = ~isLTU;
            default:   Branch = 0;
        endcase
    end
    else
            Branch = 0;
    end

endmodule
```

4.1.4 冒险检测功能电路 Hazard Detector 的设计

由前面分析可知，冒险成立的条件为：

(1). 上一条指令必须是 lw 指令（`MemRead_ex=1`）；

(2). 两条指令读写同一个寄存器(`rdAddr_ex==rs1Addr_id` 或 `rdAddr_ex==rs2Addr_id`)。

当冒险成立应清空 ID/EX 寄存器并且阻塞流水线 ID 级、IF 级流水线，所以有

$$\text{Stall} = ((\text{rdAddr_ex} == \text{rs1Addr_id}) \,||\, (\text{rdAddr_ex} == \text{rs2Addr_id})) \,\&\&\, \text{MemRead_ex}$$

$$\text{IFWrite} =\sim \text{Stall}$$

在用 Verilog HDL 描述 ID 模块时，冒险检测功能电路（Hazard Detector）等功能单元比较简单，建议直接在 ID 顶层描述。

代码中给出判定关系：

```
// 冒险检测
assign Stall = ((rdAddr_ex == rs1Addr_id) || (rdAddr_ex == rs2Addr_id)) &&
    MemRead_ex;
assign IFWrite = ~Stall;
```

4.1.5 ID 顶层设计

ID 顶层模块对指令译码、寄存器堆和分支检测等子模块进行统一封装，并完成各模块之间的信号连接。顶层模块一方面接收来自 IF/ID 流水寄存器的指令信息，另一方面接收来自 WB 阶段的寄存器写回信号。

在 ID 顶层模块中，译码得到的控制信号、寄存器读出的操作数、立即数以及分支判断结果被统一送入 ID/EX 流水寄存器，为执行阶段提供所需的数据和控制信息。

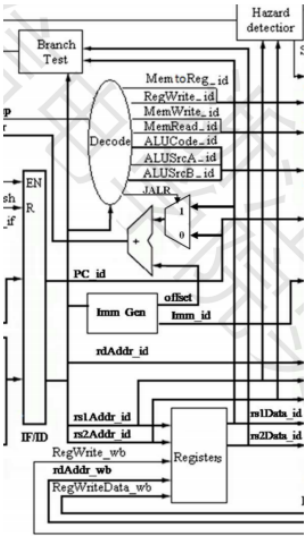


Figure 14: ID 模块原理图

根据 ID 模块原理图和以上分析，设计顶层描述为：

```
module ID(clk,Instruction_id, PC_id, RegWrite_wb, rdAddr_wb, RegWriteData_wb,
    MemRead_ex,
        rdAddr_ex, MemtoReg_id, RegWrite_id, MemWrite_id, MemRead_id, ALUCode_id,
```

```
        ALUSrcA_id, ALUSrcB_id, Stall, Branch, Jump, IFWrite, JumpAddr, Imm_id,
        rs1Data_id, rs2Data_id,rs1Addr_id,rs2Addr_id,rdAddr_id);

    input clk;
    input [31:0] Instruction_id;
    input [31:0] PC_id;
    input RegWrite_wb;
    input [4:0] rdAddr_wb;
    input [31:0] RegWriteData_wb;
    input MemRead_ex;
    input [4:0] rdAddr_ex;
    output MemtoReg_id;
    output RegWrite_id;
    output MemWrite_id;
    output MemRead_id;
    output [3:0] ALUCode_id;
    output ALUSrcA_id;
    output [1:0] ALUSrcB_id;
    output Stall;
    output Branch;
    output Jump;
    output IFWrite;
    output [31:0] JumpAddr;
    output [31:0] Imm_id;
    output [31:0] rs1Data_id;
    output [31:0] rs2Data_id;
    output [4:0] rs1Addr_id, rs2Addr_id, rdAddr_id;

    // 指令寄存器地址字段
    assign rs1Addr_id = Instruction_id[19:15];
    assign rs2Addr_id = Instruction_id[24:20];
    assign rdAddr_id = Instruction_id[11:7];

    wire JALR;
    wire [31:0] offset;
    wire [31:0] base_addr;

    // 译码单元
    Decode Decode0(
        .Instruction(Instruction_id),
        .MemtoReg(MemtoReg_id),
        .RegWrite(RegWrite_id),
        .MemWrite(MemWrite_id),
        .MemRead(MemRead_id),
        .ALUCode(ALUCode_id),
        .ALUSrcA(ALUSrcA_id),
```

```
        .ALUSrcB(ALUSrcB_id),
        .Jump(Jump),
        .JALR(JALR),
        .Imm(Imm_id),
        .offset(offset)
    );

    // 寄存器堆
    registers registers_ID(
        .clk(clk),
        .rs1Addr(rs1Addr_id),
        .rs2Addr(rs2Addr_id),
        .RegWrite(RegWrite_wb),
        .WriteAddr(rdAddr_wb),
        .WriteData(RegWriteData_wb),
        .rs1Data(rs1Data_id),
        .rs2Data(rs2Data_id)
    );

    // 分支判断
    BranchTest BranchTest0(
        .Instruction(Instruction_id),
        .rs1Data(rs1Data_id),
        .rs2Data(rs2Data_id),
        .Branch(Branch)
    );

    // 跳转基地址选择
    assign base_addr = JALR ? rs1Data_id : PC_id;

    // 跳转地址计算
    Add_32bit adder_ID(
        .a(offset),
        .b(base_addr),
        .ci(0),
        .sum(JumpAddr),
        .c_out()
    );

    // 冒险检测
    assign Stall = ((rdAddr_ex == rs1Addr_id) || (rdAddr_ex == rs2Addr_id)) &&
        MemRead_ex;
    assign IFWrite = ~Stall;

endmodule
```

4.2 EX 模块的设计

执行模块主要由 ALU 子模块、数据前推电路（Forwarding）及若干数据选择器组成。执行模块的接口信息如下表所示。

表 30.7 EX 模块的输入/输出引脚说明

引脚名称	方向	说明
ALUCode_ex[3:0]	Input	决定 ALU 采用何种运算
ALUSrcA_ex		决定 ALU 的 A 操作数的来源（rs1、PC）
ALUSrcB_ex[1:0]		决定 ALU 的 B 操作数的来源(rs2、imm 和常数 4)
Imm_ex[31:0]		立即数
rs1Addr_ex[4:0]		rs1 寄存器地址
rs2Addr_ex[4:0]		rs2 寄存器地址
rs1Data_ex[31:0]		rs1 寄存器数据
rs2Data_ex[31:0]		rs2 寄存器数据
PC_ex[31:0]		指令指针
RegWriteData_wb[31:0]		写入寄存器的数据
ALUResult_mem[31:0]		ALU 输出数据
rdAddr_mem[4:0]		寄存器的写地址
rdAddr_wb[4:0]		
RegWrite_mem		寄存器写允许信号
RegWrite_wb		
ALUResult_ex[31:0]	Output	ALU 运算结果
MemWriteData_ex[31:0]		存储器的回写数据
ALU_A [31:0]		ALU 操作数，测试时使用
ALU_B [31:0]		

Figure 15: EX 模块的引脚说明

4.2.1 ALU 子模块的设计

算术逻辑运算单元（ALU）提供 CPU 的基本运算能力，如加、减、与、或、比较、移位等。具体而言，ALU 输入为两个操作数 A、B 和控制信号 ALUCode，由控制信号 ALUCode 决定采用何种运算，运算结果为 ALUResult。整理 ALUCode 的功能表，可得到 ALU 的功能表，如下表所示。

表 30.8 ALU 的功能表

ALUCode	ALUResult
4'b0000	A + B
4'b0001	A-B
4'b0010	B
4'b0011	A&B
4'b0100	A ^ B
4'b0101	A | B
4'b0110	A << B
4'b0111	A >> B
4'b1000	A>>>B
4'b1001	A<B? 1:0，其中 A、B 为有符号数
4'b1010	A<B? 1:0，其中 A、B 为无符号数

Figure 16: ALU 功能表

ALU 需执行多种运算，为了提高运算速度，本设计可同时进行各种运算，再根据 ALUCode 信号选出所需结果。ALU 的基本结构如下图所示：

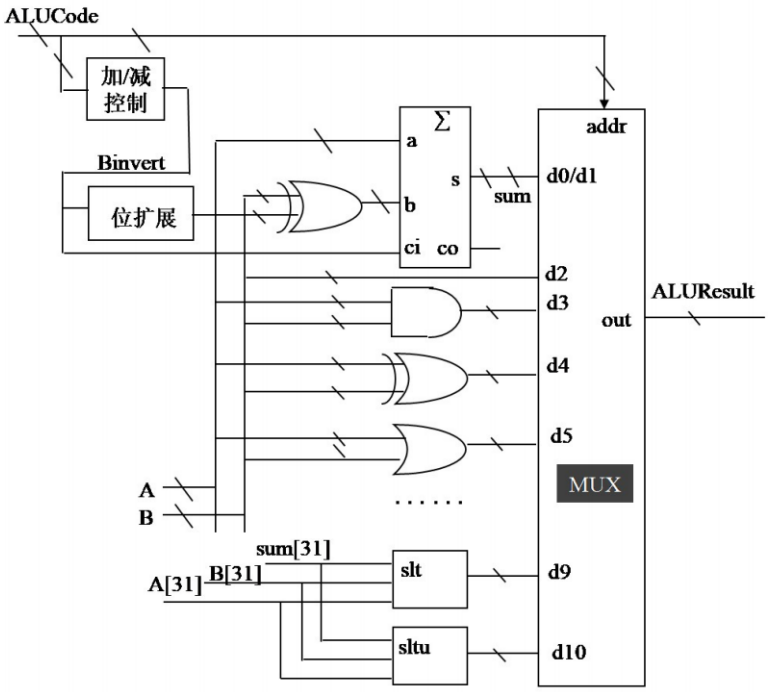


Figure 17: ALU 结构框图

ALU 设计代码如下：

```
module ALU (
   // Outputs
      ALUResult,
   // Inputs
      ALUCode, A, B);

   input [3:0] ALUCode;           // Operation select
   input [31:0]  A, B;
   output reg [31:0] ALUResult;

// Decoded ALU operation select (ALUsel) signals
  parameter   alu_add = 4'b0000;
  parameter   alu_sub = 4'b0001;
  parameter   alu_lui = 4'b0010;
  parameter   alu_and = 4'b0011;
  parameter   alu_xor = 4'b0100;
  parameter   alu_or = 4'b0101;
  parameter   alu_sll = 4'b0110;
  parameter   alu_srl = 4'b0111;
  parameter   alu_sra = 4'b1000;
  parameter   alu_slt = 4'b1001;
```

```
  parameter   alu_sltu= 4'b1010;

  wire c1, c2;
  wire [31:0] result_add, result_sub;
  wire [31:0] result_slt, result_sltu;
  wire [31:0] B_sub;
  wire signed [31:0] A_sign;

  assign A_sign = A;     // 有符号比较与算术右移使用
  assign B_sub = ~B;     // 用于减法运算

  // 加法与减法共用加法器
  Add_32bit ADD_alu1(.a(A), .b(B), .ci(0), .sum(result_add), .c_out(c1));
  Add_32bit ADD_alu2(.a(A), .b(B_sub), .ci(1), .sum(result_sub), .c_out(c2));

  always @(*) begin
    case (ALUCode)
      alu_add: ALUResult = result_add;
      alu_sub: ALUResult = result_sub;
      alu_lui: ALUResult = B;
      alu_and: ALUResult = A & B;
      alu_xor: ALUResult = A ^ B;
      alu_or:  ALUResult = A | B;
      alu_sll: ALUResult = A << B;
      alu_srl: ALUResult = A >> B;
      alu_sra: ALUResult = A_sign >>> B;
      alu_slt: ALUResult = (c2 == 1) ? 1 : 0; // 有符号小于判断
      alu_sltu: begin
                if (A < B) ALUResult = 1;
                else      ALUResult = 0;
              end
      default: ALUResult = result_add;
    endcase
  end

endmodule
```

4.2.2 EX 顶层及数据前推电路的设计

操作数 A 和 B 分别由数据选择器决定，数据选择器地址信号 ForwardA、ForwardB 的含义如下表所示。

则给出 EX 顶层代码（包括数据前推）如下：

```
module EX(ALUCode_ex, ALUSrcA_ex, ALUSrcB_ex,Imm_ex, rs1Addr_ex, rs2Addr_ex,
    rs1Data_ex,
```

表 30.9 前推电路输出信号的含义

地 址	操作数来源	说 明
ForwardA= 2'b00	rs1Data_ex	操作数 A 来自寄存器堆
ForwardA=2'b01	RegWriteData_wb	操作数 A 来自二阶数据相关的转发数据
ForwardA= 2'b10	ALUResult_mem	操作数 A 来自一阶数据相关的转发数据
ForwardB= 2'b00	rs2Data_ex	操作数 B 来自寄存器堆
ForwardB= 2'b01	RegWriteData_wb	操作数 B 来自二阶数据相关的转发数据
ForwardB=2'b10	ALUResult_mem	操作数 B 来自一阶数据相关的转发数据

Figure 18: 前推电路输出信号的含义

```
        rs2Data_ex, PC_ex, RegWriteData_wb, ALUResult_mem,rdAddr_mem, rdAddr_wb,
        RegWrite_mem, RegWrite_wb, ALUResult_ex, MemWriteData_ex, ALU_A, ALU_B);
    input [3:0] ALUCode_ex;
    input ALUSrcA_ex;
    input [1:0]ALUSrcB_ex;
    input [31:0] Imm_ex;
    input [4:0] rs1Addr_ex;
    input [4:0] rs2Addr_ex;
    input [31:0] rs1Data_ex;
    input [31:0] rs2Data_ex;
    input [31:0] PC_ex;
    input [31:0] RegWriteData_wb;
    input [31:0] ALUResult_mem;
    input [4:0]rdAddr_mem;
    input [4:0] rdAddr_wb;
    input RegWrite_mem;
    input RegWrite_wb;
    output [31:0] ALUResult_ex;
    output [31:0] MemWriteData_ex;
    output [31:0] ALU_A;
    output [31:0] ALU_B;

    wire[1:0] ForwardA, ForwardB; // 前递控制信号
    wire [31:0] A; // 前递后的 rs1 数据
    wire [31:0] B; // 前递后的 rs2 数据

    // 前递控制逻辑（解决数据相关）
    assign ForwardA[0] = RegWrite_wb && (rdAddr_wb!=0) && (rdAddr_mem!=rs1Addr_ex) && (
        rdAddr_wb==rs1Addr_ex);
    assign ForwardA[1] = RegWrite_mem && (rdAddr_mem!=0) && (rdAddr_mem==rs1Addr_ex);
    assign ForwardB[0] = RegWrite_wb && (rdAddr_wb!=0) && (rdAddr_mem!=rs2Addr_ex) && (
        rdAddr_wb==rs2Addr_ex);
    assign ForwardB[1] = RegWrite_mem && (rdAddr_mem!=0) && (rdAddr_mem==rs2Addr_ex);

    // 前递多路选择器
    mux mux_ex1(.a(rs1Data_ex), .b(RegWriteData_wb), .c(ALUResult_mem), .select(
```

```
        ForwardA), .result(A));
    mux mux_ex2(.a(rs2Data_ex), .b(RegWriteData_wb), .c(ALUResult_mem), .select(
        ForwardB), .result(B));

    // 写入数据存储器的数据
    assign MemWriteData_ex = B;

    // ALU 第一个操作数选择
    assign ALU_A = ALUSrcA_ex? PC_ex:A;

    // ALU 第二个操作数选择
    mux mux_ex3(.a(B), .b(Imm_ex), .c(32'd4), .select(ALUSrcB_ex), .result(ALU_B));

    // ALU 运算
    ALU ALU_ex(.ALUCode(ALUCode_ex), .A(ALU_A), .B(ALU_B), .ALUResult(ALUResult_ex));

endmodule
```

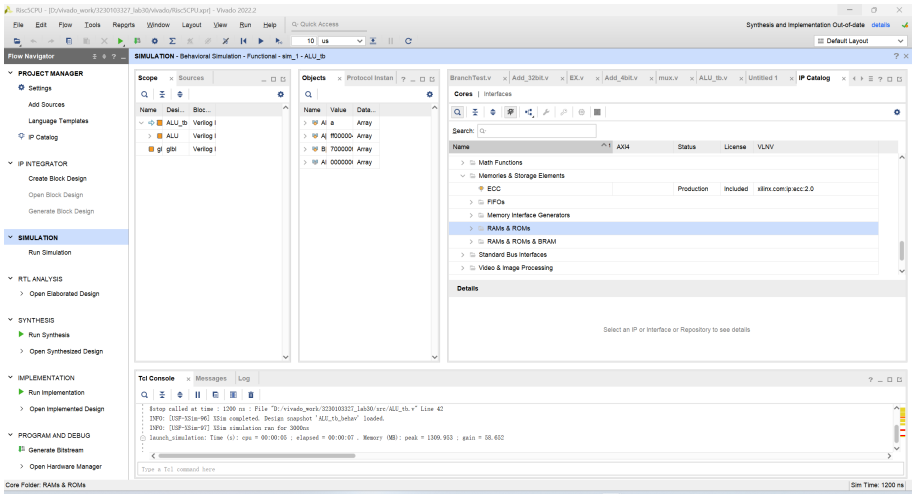
4.3 DataRAM 模块的设计

数据存储器可采用 Xilinx 提供的 IP 内核实现。考虑到 FPGA 片上资源的限制，数据存储器设计为容量为 64 × 32 bit 的单端口 RAM，其输出方式采用组合输出（Non Registered）。

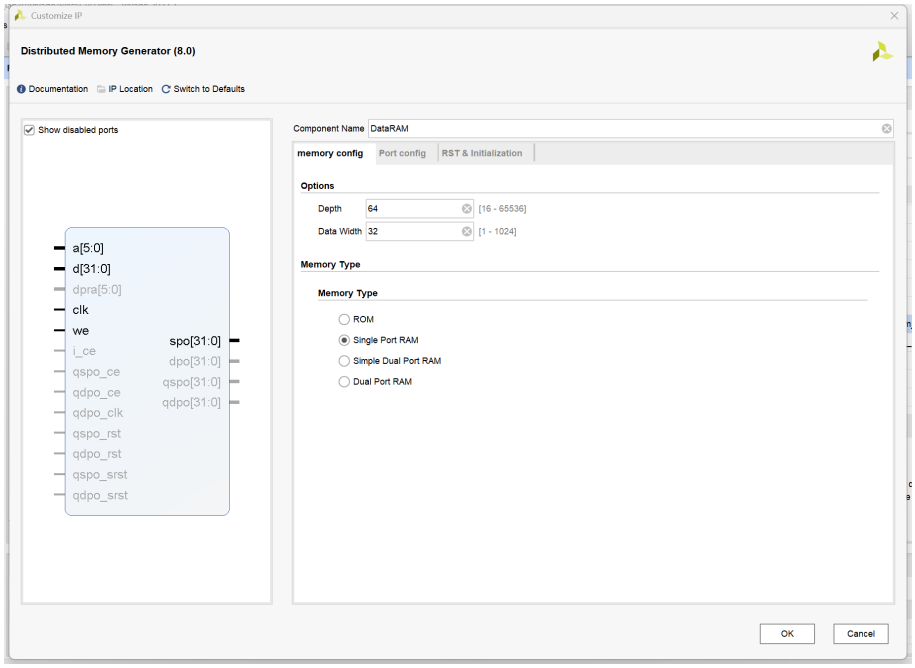
由于数据存储器容量为 64 × 32 bit，因此存储器地址宽度为 6 位，并与 ALUResult_mem[7:2] 相连接。

数据存储器的设计步骤如下：

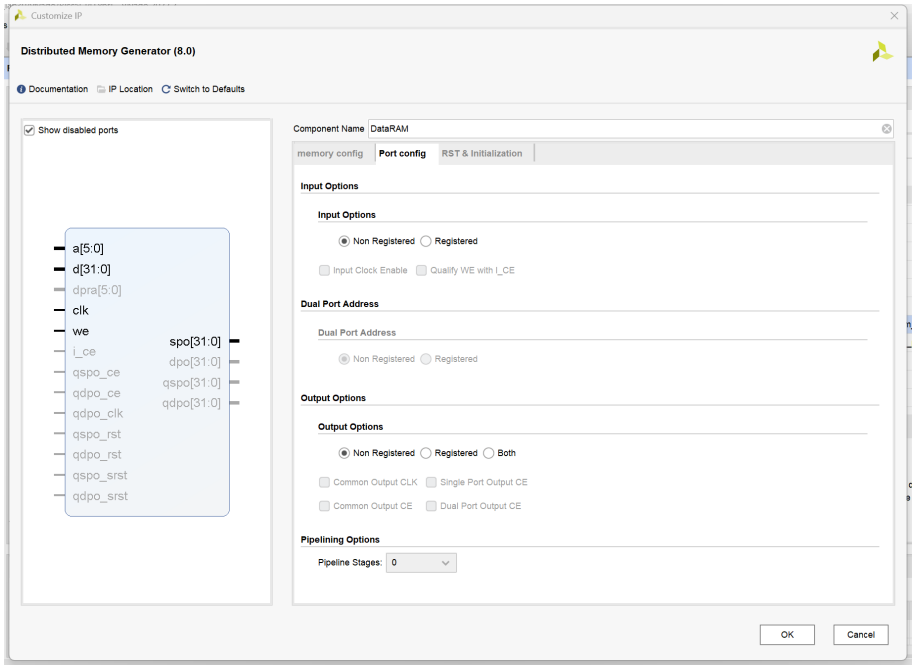
(1). 在 PROJECT MANAGER 栏中的 IP Catalog 中，依次选择 RAMs & ROMs，并点击 Distributed Memory Generator，进入 IP 核的配置界面。



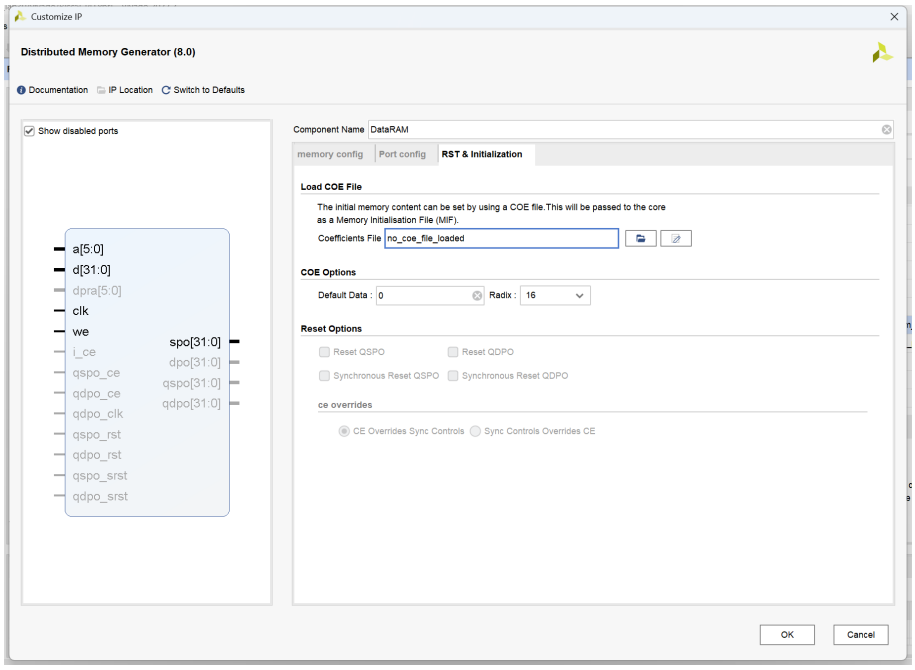
(2). 在第一列 Memory Config 中进行参数设置。在 Options 选项中，将存储深度（Depth）设置为 64，数据位宽（Data Width）设置为 32 bit；在 Memory Type 中选择 Single Port RAM。



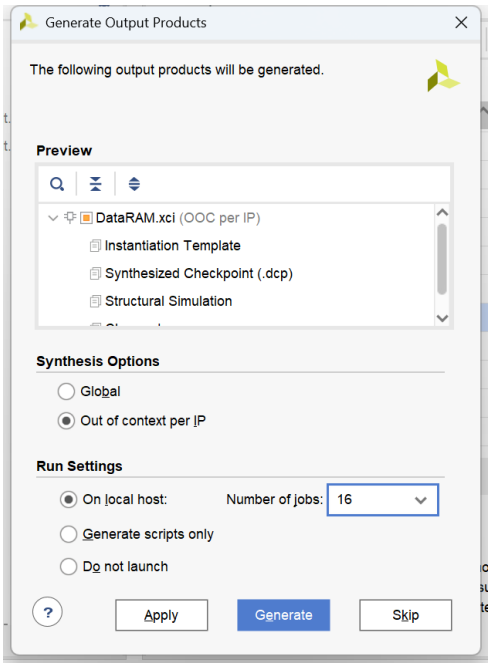
(3). 在第二列 Port Config 中进行端口配置，在 Input Options 和 Output Options 中均选择 Non Registered，以实现组合方式的数据输出。



(4). 在第三列 `RST & Initialization` 中设置模块名为 `DataRam`。



(5). 最后在界面内如下图设置, 设置 number of jobs 的值为 16。



4.4 IF 模块的设计

IF 模块主要由指令指针寄存器（PC）、指令存储器子模块（Instruction ROM）、指令指针选择器（MUX）以及一个 32 位加法器组成。其接口信息如下图所示。

表 30.10 IF 模块的输入/输出引脚说明

引脚名称	方向	说明
clk	Input	系统时钟
reset		系统复位信号，高电平有效
Branch		条件分支指令的条件判断结果
Jump		无条件分支指令的条件判断结果
IFWrite		流水线阻塞信号
JumpAddr[31:0]		分支地址
Instruction [31:0]	Output	指令机器码
IF_flush		流水线清空信号
PC [31:0]		PC 值

Figure 19: IF 模块接口信息

指令存储器采用组合存储结构，可使用 Verilog HDL 设计为查找表形式的 ROM。考虑到 FPGA 资源限制，该指令存储器容量设计为 64 × 32 bit。

实验中，作者提供了指令存储器模块 InstructionROM.v，ROM 中预先存放一段用于功能验证的简单测试程序机器码，对应的测试程序如下所示：

```
       lui X30, 0x3000
       jalr X31, later(X0)
earlier:sw X28, 0x0C(X0)
       lw X29, 4(X6)
       slliX5, X29, 2 //数据冒险 lw X28, 4(X6)
       sltuX28,X6,X7 done:jal X31, done
later: bneX0, X0, end // 分支条件不成立
       addi X5, X30, 0x42 add X6, X0, X31
       sub X7, X5, X6 //操作A二阶数据相关，操作B一阶数据相关
       or X28, X7, X5 //操作A一阶数据相关，操作B三阶数据相关
       beq X0, X0, earlier // 分支条件成立
end: nop
```

图 19 为 IF 模块结构图：

结合数据通路的相关原理以及图 19 所示结构，IF 模块的工作流程如下。

在不发生跳转的情况下，即 Branch 和 Jump 信号均为无效时，程序计数器 PC 保持顺序执行。模块从当前 PC 所指向的指令存储器地址中取出指令，完成取指操作后，通过加法器实现 PC+4，作为下一条指令的地址。

当发生分支或跳转时，即 Branch 或 Jump 信号有效，需要改变指令执行的顺序。此时，控制逻辑通过多路选择器将下一条 PC 的值选择为 JumpAddr，即跳转目标地址。同时，由于跳转会使得顺序预取的指令失效，因此需要对 IF/ID 流水线寄存器进行清空，该过程通过生成 IF_flush 信号实现。

此外，当发生流水线阻塞时，IFWrite 信号被置为无效，在该时钟周期内程序计数器 PC 保持不变，从而暂停 IF 级取指操作，等待后续流水线阶段解除阻塞。

IF 模块设计代码如下：

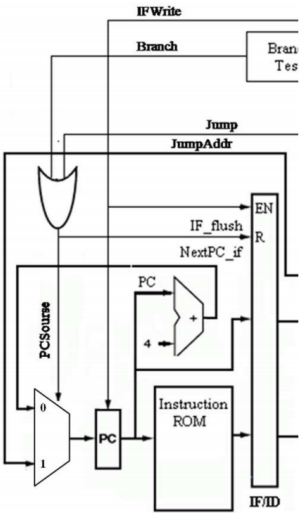


Figure 20: IF 模块结构图

```
module IF(clk, reset, Branch,Jump, IFWrite, JumpAddr,Instruction_if,PC,IF_flush);
    input clk;
    input reset;
    input Branch;
    input Jump;
    input IFWrite;
    input [31:0] JumpAddr;
    output [31:0] Instruction_if;
    output [31:0] PC;
    output IF_flush;

    wire[31:0] NextPC_if, PCSource;
    reg[31:0] PC;

    // 分支或跳转时刷新 IF/ID
    assign IF_flush = Branch || Jump;

    // 下一条 PC 选择
    assign PCSource = IF_flush ? JumpAddr : NextPC_if;

    always@(posedge clk)
    begin
        if(reset)
            PC <= 32'd0;
        else if(IFWrite)
            PC <= PCSource;
        else
            PC <= PC;
    end

```

```
    // PC + 4
    Add_32bit adder_IF(.a(PC), .b(32'd4), .ci(0), .sum(NextPC_if), .c_out());

    // 指令存储器读取
    InstructionROM InstROM0(.addr(PC[7:2]), .dout(Instruction_if));

endmodule
```

4.5 流水线寄存器的设计

流水线寄存器负责将流水线的各部分分开，共有 IF/ID、ID/EX、EX/MEM、MEM/WB 四组，对四组流水线寄存器要求不完全相同，因此设计也有不同考虑。

4.5.1 IF/ID 流水线寄存器

当流水线发生数据冒险时，需要阻塞 IF/ID 流水线寄存器；若跳转指令或分支成立，则还需要清空 ID/EX 流水线寄存器。因此，IF/ID 流水线寄存器除同步清零功能外，还需要具有保持功能（即具有使能 EN 信号输入）。

```
module IFID(
    input clk,
    input en,
    input reset,
    input[31:0] PC_if,
    input[31:0] Instruction_if,

    output reg [31:0]PC_id,
    output reg [31:0]Instruction_id
    );

    always@(posedge clk)
    begin
        if(reset)
        begin
            PC_id <= 0;
            Instruction_id <= 0;
        end
        else if(en)
        begin
            PC_id <= PC_if;
            Instruction_id <= Instruction_if;
        end
        else
        begin
```

```
            PC_id <= PC_id;
            Instruction_id <= Instruction_id;
        end
    end

endmodule
```

4.5.2 ID/EX 流水线寄存器

当流水线发生数据冒险时，需要清空 ID/EX 流水线寄存器而插入一个气泡，因此 ID/EX 流水线寄存器是一个带同步清零功能的 D 型寄存器。

```
module IDEX(
 // Inputs
    clk, reset, MemtoReg_id, RegWrite_id, MemWrite_id, MemRead_id, ALUCode_id,
        ALUSrcA_id, ALUSrcB_id,
    PC_id, Imm_id, rdAddr_id, rs1Addr_id, rs2Addr_id, rs1Data_id, rs2Data_id,
    // Outputs
    MemtoReg_ex, RegWrite_ex, MemWrite_ex, MemRead_ex, ALUCode_ex, ALUSrcA_ex,
        ALUSrcB_ex,
    PC_ex, Imm_ex, rdAddr_ex, rs1Addr_ex, rs2Addr_ex, rs1Data_ex, rs2Data_ex);

    input clk, reset;
    input MemtoReg_id, RegWrite_id, MemWrite_id, MemRead_id;
    input[3:0] ALUCode_id;
    input ALUSrcA_id;
    input[1:0] ALUSrcB_id;
    input[4:0] rdAddr_id, rs1Addr_id, rs2Addr_id;
    input[31:0] rs1Data_id, rs2Data_id, PC_id, Imm_id;

    output reg MemtoReg_ex, RegWrite_ex, MemWrite_ex, MemRead_ex;
    output reg [3:0] ALUCode_ex;
    output reg ALUSrcA_ex;
    output reg [1:0] ALUSrcB_ex;
    output reg [4:0] rdAddr_ex, rs1Addr_ex, rs2Addr_ex;
    output reg [31:0] rs1Data_ex, rs2Data_ex, PC_ex, Imm_ex;

    always@(posedge clk)
    begin
        if(reset) begin
            MemtoReg_ex <= 0;
            RegWrite_ex <= 0;
            MemWrite_ex <= 0;
            MemRead_ex <= 0;
            ALUCode_ex <= 0;
```

```
            ALUSrcA_ex <= 0;
            ALUSrcB_ex <= 0;
            PC_ex      <= 0;
            Imm_ex     <= 0;
            rdAddr_ex  <= 0;
            rs1Addr_ex <= 0;
            rs2Addr_ex <= 0;
            rs1Data_ex <= 0;
            rs2Data_ex <= 0;
        end
        else begin
            MemtoReg_ex <= MemtoReg_id;
            RegWrite_ex <= RegWrite_id;
            MemWrite_ex <= MemWrite_id;
            MemRead_ex <= MemRead_id;
            ALUCode_ex <= ALUCode_id;
            ALUSrcA_ex <= ALUSrcA_id;
            ALUSrcB_ex <= ALUSrcB_id;
            PC_ex      <= PC_id;
            Imm_ex     <= Imm_id;
            rdAddr_ex  <= rdAddr_id;
            rs1Addr_ex <= rs1Addr_id;
            rs2Addr_ex <= rs2Addr_id;
            rs1Data_ex <= rs1Data_id;
            rs2Data_ex <= rs2Data_id;
        end
    end

endmodule
```

4.5.3 EX/MEM 流水线寄存器

EX/MEM 流水线寄存器是普通的 D 型寄存器。

```
module EXMEM(
    // Inputs
    clk, MemtoReg_ex, RegWrite_ex, MemWrite_ex, ALUResult_ex, MemWriteData_ex,
        rdAddr_ex,
    // Outputs
    MemtoReg_mem, RegWrite_mem, MemWrite_mem, ALUResult_mem, MemWriteData_mem,
        rdAddr_mem);

    input clk;
    input MemtoReg_ex, RegWrite_ex, MemWrite_ex;
    input[31:0] ALUResult_ex, MemWriteData_ex;
    input[4:0] rdAddr_ex;
```

```

    output reg MemtoReg_mem, RegWrite_mem, MemWrite_mem;
    output reg [31:0] ALUResult_mem, MemWriteData_mem;
    output reg [4:0] rdAddr_mem;

    always@(posedge clk)
    begin
        MemtoReg_mem     <= MemtoReg_ex;
        RegWrite_mem     <= RegWrite_ex;
        MemWrite_mem     <= MemWrite_ex;
        ALUResult_mem    <= ALUResult_ex;
        MemWriteData_mem <= MemWriteData_ex;
        rdAddr_mem       <= rdAddr_ex;
    end

endmodule
```

4.5.4 MEM/WB 流水线寄存器

MEM/WB 流水线寄存器是普通的 D 型寄存器。

```
module MEMWB(
     // Inputs
    clk, MemtoReg_mem, RegWrite_mem, MemDout_mem, ALUResult_mem, rdAddr_mem,
    // Outputs
    MemtoReg_wb, RegWrite_wb, MemDout_wb, ALUResult_wb, rdAddr_wb);
    input clk;
    input MemtoReg_mem, RegWrite_mem;
    input[31:0] MemDout_mem, ALUResult_mem;
    input[4:0] rdAddr_mem;

    output reg MemtoReg_wb, RegWrite_wb;
    output reg [31:0] MemDout_wb, ALUResult_wb;
    output reg [4:0] rdAddr_wb;

    always@(posedge clk)
    begin
        MemtoReg_wb   <= MemtoReg_mem;
        RegWrite_wb   <= RegWrite_mem;
        MemDout_wb    <= MemDout_mem;
        ALUResult_wb  <= ALUResult_mem;
        rdAddr_wb     <= rdAddr_mem;
    end

endmodule
```

4.6 顶层文件的设计

按照原理框图连接各模块即可。为了测试方便，可将关键变量输出，关键变量有：

- 指令指针 PC；
- 指令码 Instruction_id；
- 流水线插入气泡标志 Stall；
- 分支标志 JumpFlag，即 {Jump, Branch}；
- ALU 输入输出（ALU_A、ALU_B、ALUResult_ex）；
- 数据存储器的输出 MemDout_mem。

顶层文件的代码设计为：

```
module Risc5CPU(clk, reset, JumpFlag, Instruction_id, ALU_A,
                  ALU_B, ALUResult_ex, PC, MemDout_mem,Stall);
   input clk;
   input reset;
   output[1:0] JumpFlag;
   output [31:0] Instruction_id;
   output [31:0] ALU_A;
   output [31:0] ALU_B;
   output [31:0] ALUResult_ex;
   output [31:0] PC;
   output [31:0] MemDout_mem;
   output Stall;

   wire IFWrite, IF_flush;
   wire[31:0] JumpAddr;
   wire[31:0] Instruction_if, PC;

   wire MemtoReg_id, RegWrite_id, MemWrite_id, MemRead_id;
   wire[3:0] ALUCode_id;
   wire ALUSrcA_id;
   wire[1:0] ALUSrcB_id;
   wire[4:0] rdAddr_id, rs1Addr_id, rs2Addr_id;
   wire[31:0] rs1Data_id, rs2Data_id, PC_id, Imm_id;

   wire MemtoReg_ex, RegWrite_ex, MemWrite_ex, MemRead_ex;
   wire [3:0] ALUCode_ex;
   wire ALUSrcA_ex;
   wire [1:0] ALUSrcB_ex;
   wire [4:0] rdAddr_ex, rs1Addr_ex, rs2Addr_ex;
   wire [31:0] rs1Data_ex, rs2Data_ex, PC_ex, Imm_ex, MemWriteData_ex, ALUResult_ex;
```

```

    wire MemtoReg_mem, RegWrite_mem, MemWrite_mem;
    wire [31:0] ALUResult_mem, MemWriteData_mem;
    wire [4:0] rdAddr_mem;

    wire MemtoReg_wb, RegWrite_wb;
    wire [31:0] MemDout_wb, ALUResult_wb, RegWriteData_wb;
    wire [4:0] rdAddr_wb;


    //========================
    // IF & IF/ID
    //========================
    IF IF_1(
        .clk(clk),
        .reset(reset),
        .Branch(JumpFlag[0]),
        .Jump(JumpFlag[1]),
        .IFWrite(IFWrite),
        .JumpAddr(JumpAddr),
        .Instruction_if(Instruction_if),
        .PC(PC),
        .IF_flush(IF_flush)
    );

    IFID IF_ID_1(
        .clk(clk),
        .en(IFWrite),
        .reset(IF_flush|reset),
        .PC_if(PC),
        .Instruction_if(Instruction_if),
        .PC_id(PC_id),
        .Instruction_id(Instruction_id)
    );


    //========================
    // ID
    //========================
    ID ID_1(
        .clk(clk),
        .Instruction_id(Instruction_id),
        .PC_id(PC_id),
        .RegWrite_wb(RegWrite_wb),
        .rdAddr_wb(rdAddr_wb),
        .RegWriteData_wb(RegWriteData_wb),
```

```
        .MemRead_ex(MemRead_ex),
        .rdAddr_ex(rdAddr_ex),
        .MemtoReg_id(MemtoReg_id),
        .RegWrite_id(RegWrite_id),
        .MemWrite_id(MemWrite_id),
        .MemRead_id(MemRead_id),
        .ALUCode_id(ALUCode_id),
        .ALUSrcA_id(ALUSrcA_id),
        .ALUSrcB_id(ALUSrcB_id),
        .Stall(Stall),
        .Branch(JumpFlag[0]),
        .Jump(JumpFlag[1]),
        .IFWrite(IFWrite),
        .JumpAddr(JumpAddr),
        .Imm_id(Imm_id),
        .rs1Data_id(rs1Data_id),
        .rs2Data_id(rs2Data_id),
        .rs1Addr_id(rs1Addr_id),
        .rs2Addr_id(rs2Addr_id),
        .rdAddr_id(rdAddr_id)
    );


    //========================
    // ID/EX & EX
    //========================
    IDEX ID_EX_1(
        .clk(clk),
        .reset(Stall|reset),
        .MemtoReg_id(MemtoReg_id),
        .RegWrite_id(RegWrite_id),
        .MemWrite_id(MemWrite_id),
        .MemRead_id(MemRead_id),
        .ALUCode_id(ALUCode_id),
        .ALUSrcA_id(ALUSrcA_id),
        .ALUSrcB_id(ALUSrcB_id),
        .PC_id(PC_id),
        .Imm_id(Imm_id),
        .rdAddr_id(rdAddr_id),
        .rs1Addr_id(rs1Addr_id),
        .rs2Addr_id(rs2Addr_id),
        .rs1Data_id(rs1Data_id),
        .rs2Data_id(rs2Data_id),
        .MemtoReg_ex(MemtoReg_ex),
        .RegWrite_ex(RegWrite_ex),
        .MemWrite_ex(MemWrite_ex),
```

```
        .MemRead_ex(MemRead_ex),
        .ALUCode_ex(ALUCode_ex),
        .ALUSrcA_ex(ALUSrcA_ex),
        .ALUSrcB_ex(ALUSrcB_ex),
        .PC_ex(PC_ex),
        .Imm_ex(Imm_ex),
        .rdAddr_ex(rdAddr_ex),
        .rs1Addr_ex(rs1Addr_ex),
        .rs2Addr_ex(rs2Addr_ex),
        .rs1Data_ex(rs1Data_ex),
        .rs2Data_ex(rs2Data_ex)
    );

    EX EX_1(
        .ALUCode_ex(ALUCode_ex),
        .ALUSrcA_ex(ALUSrcA_ex),
        .ALUSrcB_ex(ALUSrcB_ex),
        .Imm_ex(Imm_ex),
        .rs1Addr_ex(rs1Addr_ex),
        .rs2Addr_ex(rs2Addr_ex),
        .rs1Data_ex(rs1Data_ex),
        .rs2Data_ex(rs2Data_ex),
        .PC_ex(PC_ex),
        .RegWriteData_wb(RegWriteData_wb),
        .ALUResult_mem(ALUResult_mem),
        .rdAddr_mem(rdAddr_mem),
        .rdAddr_wb(rdAddr_wb),
        .RegWrite_mem(RegWrite_mem),
        .RegWrite_wb(RegWrite_wb),
        .ALUResult_ex(ALUResult_ex),
        .MemWriteData_ex(MemWriteData_ex),
        .ALU_A(ALU_A),
        .ALU_B(ALU_B)
    );


    //========================
    // EX/MEM & MEM
    //========================
    EXMEM EX_MEM_1(
        .clk(clk),
        .MemtoReg_ex(MemtoReg_ex),
        .RegWrite_ex(RegWrite_ex),
        .MemWrite_ex(MemWrite_ex),
        .ALUResult_ex(ALUResult_ex),
        .MemWriteData_ex(MemWriteData_ex),
```

```
        .rdAddr_ex(rdAddr_ex),
        .MemtoReg_mem(MemtoReg_mem),
        .RegWrite_mem(RegWrite_mem),
        .MemWrite_mem(MemWrite_mem),
        .ALUResult_mem(ALUResult_mem),
        .MemWriteData_mem(MemWriteData_mem),
        .rdAddr_mem(rdAddr_mem)
    );

    DataRAM DataRAM_1(
        .a(ALUResult_mem[7:2]),
        .d(MemWriteData_mem),
        .clk(clk),
        .we(MemWrite_mem),
        .spo(MemDout_mem)
    );


    //========================
    // MEM/WB & WB
    //========================
    MEMWB MEM_WB_1(
        .clk(clk),
        .MemtoReg_mem(MemtoReg_mem),
        .RegWrite_mem(RegWrite_mem),
        .MemDout_mem(MemDout_mem),
        .ALUResult_mem(ALUResult_mem),
        .rdAddr_mem(rdAddr_mem),
        .MemtoReg_wb(MemtoReg_wb),
        .RegWrite_wb(RegWrite_wb),
        .MemDout_wb(MemDout_wb),
        .ALUResult_wb(ALUResult_wb),
        .rdAddr_wb(rdAddr_wb)
    );

    // 写回数据选择
    assign RegWriteData_wb = MemtoReg_wb ? MemDout_wb : ALUResult_wb;

endmodule
```

5 仿真与结果分析

（1）Decode 模块仿真结果及分析

Decode 仿真结果如图所示：

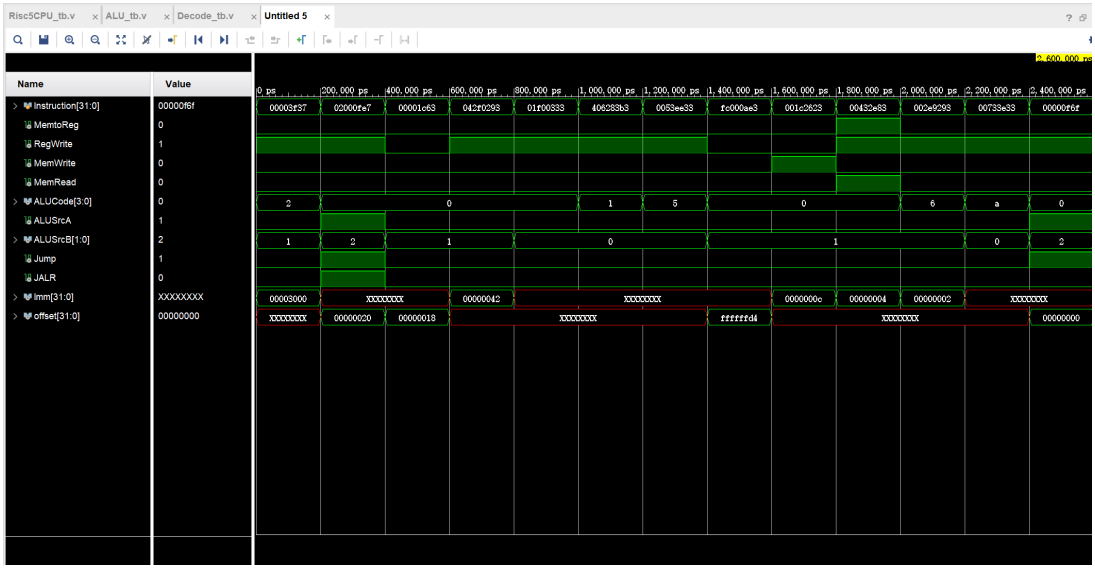


Figure 21: Decode 仿真结果图

对照该表格，发现各指令对应输出的 MemRead、MemtoReg、MemWrite、RegWrite、Jump、JALR、ALUSrcA、ALUSrcB、ALUCode 信号与输出的 Imm、offset 均正确。在 Imm、offset 不需要输出时，对应结果为方针图中的红色高阻态，输出随机数。综上，Decode 模块设计正确。

Instruction	MemRead	MemtoReg	MemWrite	RegWrite	Jump	JALR	ALUSrcA	ALUSrcB	ALUCode	Imm	offset
lui X30, 0x3000	0	0	0	1	0	0	0	2'h1	4'h2	32'h00003000	-
jalr X31, later(X0)	0	0	0	1	1	1	1	2'h2	4'h0	-	32'h00000020
bne X0, X0	0	0	0	0	0	0	0	2'h1	4'h0	-	32'h00000018
addi X5, X30, 42	0	0	0	1	0	0	0	2'h1	4'h0	32'h00000042	-
add X6, X0, X31	0	0	0	1	0	0	0	2'h0	4'h0	-	-
sub X7, X5, X6	0	0	0	1	0	0	0	2'h0	4'h1	-	-
or X28, X7, X5	0	0	0	1	0	0	0	2'h0	4'h5	-	-
beq X0, X0, earlier	0	0	0	0	0	0	0	2'h1	4'h0	-	32'hffffffd4
sw X28, 0C(X0)	0	0	1	0	0	0	0	2'h1	4'h0	32'h0000000c	-
lw X29, 04(X6)	1	1	0	1	0	0	0	2'h1	4'h0	32'h00000004	-
sll X5, X29	0	0	0	1	0	0	0	2'h1	4'h6	32'h00000002	-
sltu X28, X6, X7	0	0	0	1	0	0	0	2'h0	4'ha	-	-
jal X31, done	0	0	0	1	1	0	0	2'h1	4'h2	-	32'h00000000

Table 1: 各类指令对应的控制信号与立即数

（2）ALU 模块仿真结果及分析

ALU 仿真结果如图所示：

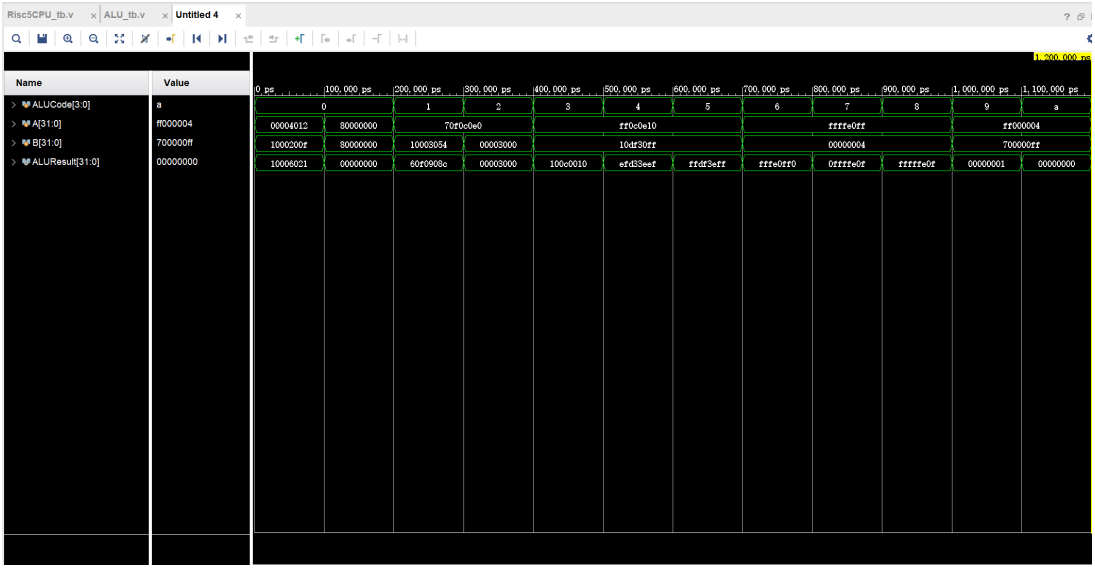


Figure 22: ALU 仿真结果图

与 ALU 参照表对照，发现 ALUcode、A、B、Result 均正确，则 ALU 设计符合预期。

A	ALUCode	B	ALUResult
32'h00004012	0(+)	32'h1000200f	32'h10006021
32'h80000000	0(+)	32'h80000000	32'h00000000
32'h70f0c0e0	1(-)	32'h10003054	32'h60f0908c
32'h70f0c0e0	2(B)	32'h00003000	32'h00003000
32'hff0c0e10	3(&)	32'h10df30ff	32'h100c0010
32'hff0c0e10	4(^)	32'h10df30ff	32'hefd33eef
32'hff0c0e10	5(|)	32'h10df30ff	32'hffdf3eff
32'hffffe0ff	6(<<)	32'h00000004	32'hfffe0ff0
32'hffffe0ff	7(>>)	32'h00000004	32'h0ffffe0f
32'hffffe0ff	8(>>>)(算数右移)	32'h00000004	32'hfffffe0f (保持符号位不变)
32'hff00000004	9(A<B?1:0)(有符号数)	32'h700000ff	32'h00000001 (此时 A 为负数, B 为正数)
32'hff00000004	10(A<B?1:0)（无符号数)	32'h700000ff	32'h00000000

Figure 23: ALU 结果参照表

（3）IF 模块仿真结果及分析

IF 模块仿真结果如图所示：

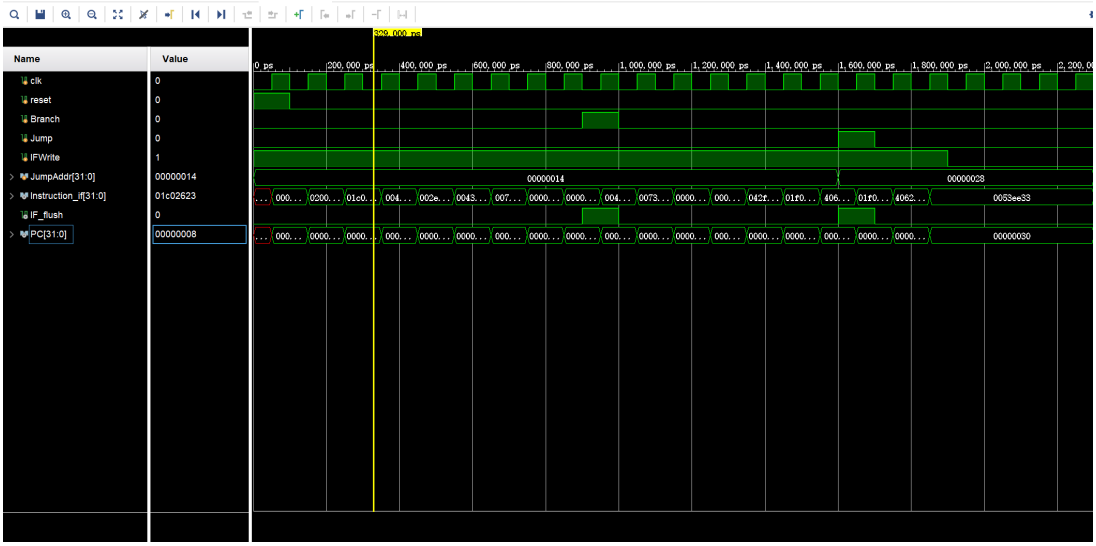


Figure 24: IF 仿真结果图

IF 仿真结果放大对照图如下所示：

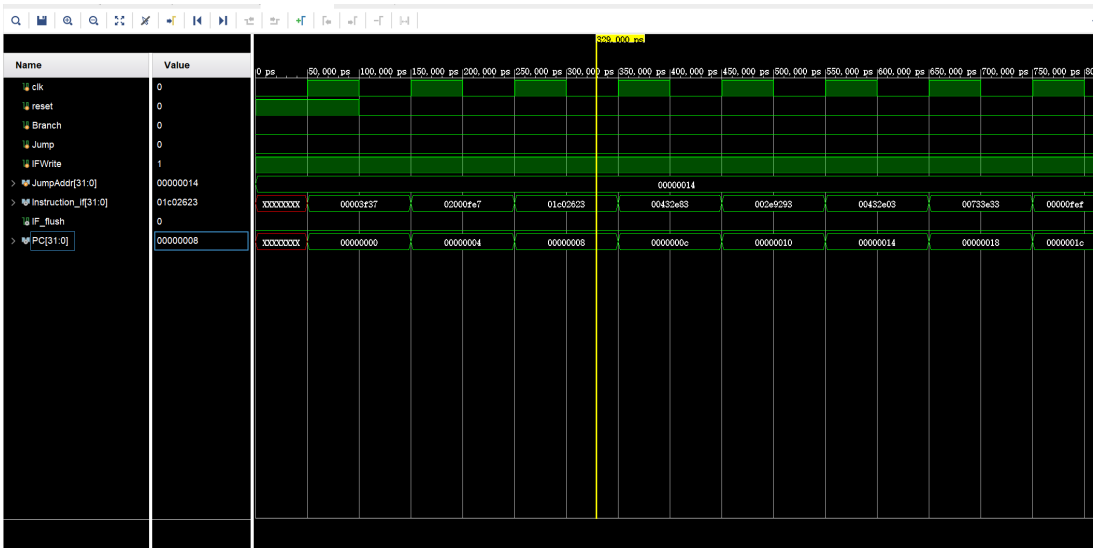


Figure 25: IF 仿真结果放大图 1

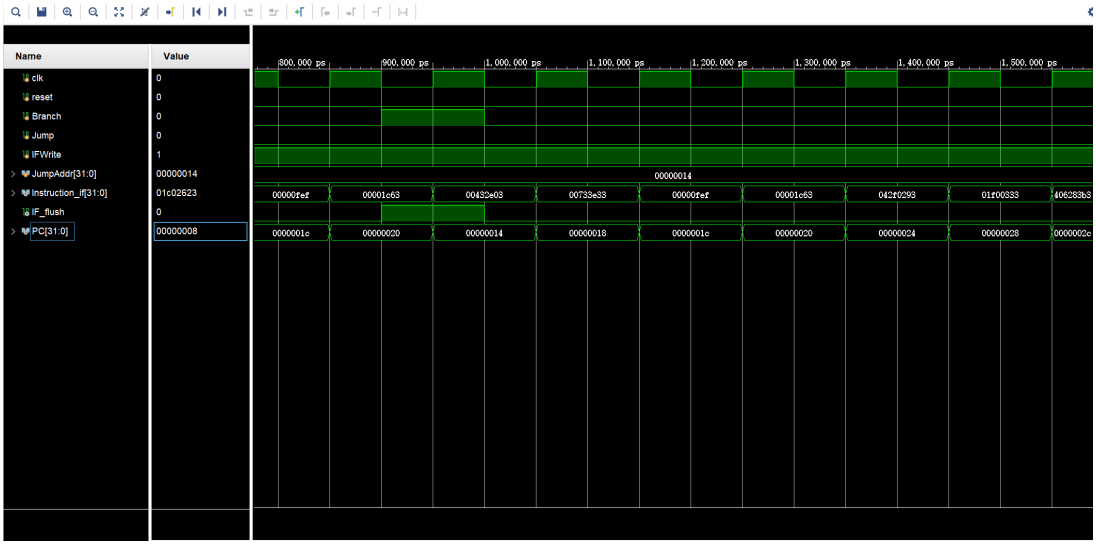


Figure 26: IF 仿真结果放大图 2

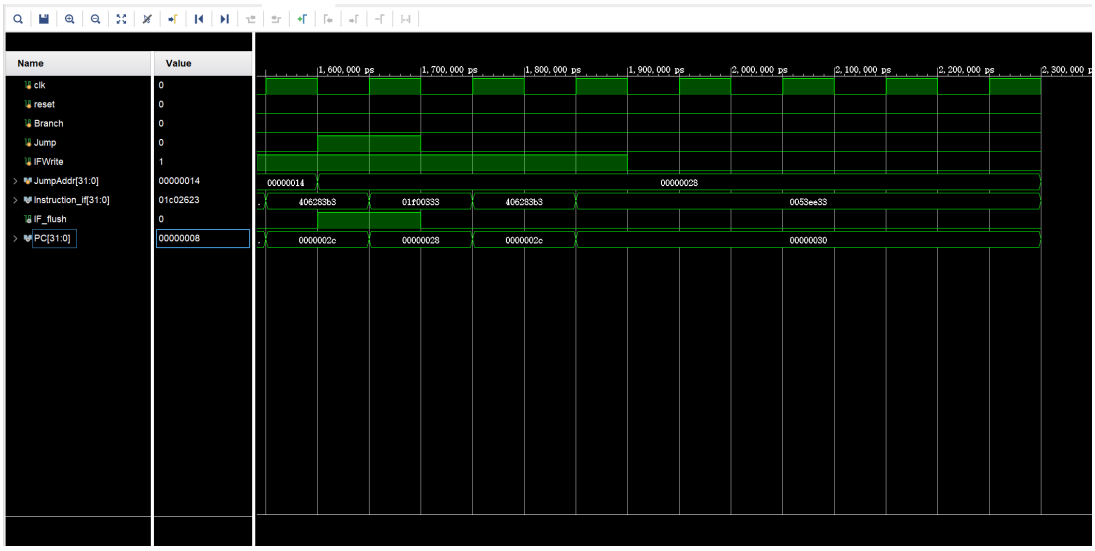


Figure 27: IF 仿真结果放大图 3

从图中可以看出，在无分支或跳转发生时，程序计数器 PC 在每个时钟周期按 PC=PC+4 的方式递增；当 `IF_flush`=1 时，下一个时钟周期 PC 被更新为 `JumpAddr`，说明 IF 模块能够在发生分支或跳转时正确刷新流水线并跳转到目标地址执行。则 IF 模块设计符合预期。

(4) RISC-V CPU 仿真结果分析

CPU 顶层模块仿真结果如下图所示：

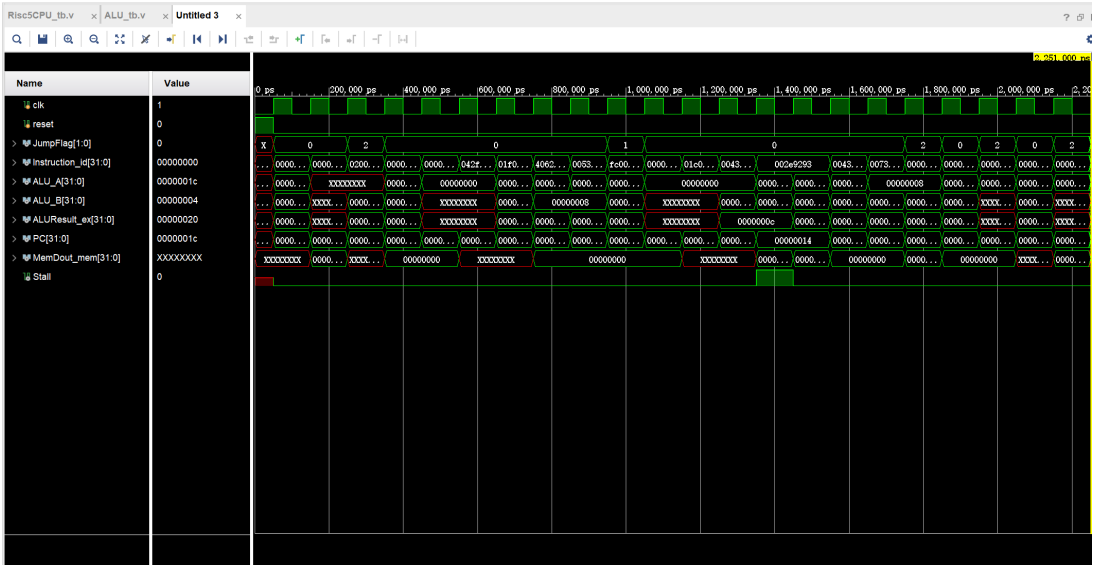


Figure 28: CPU 仿真结果图

CPU 仿真结果放大对照图如下图所示：

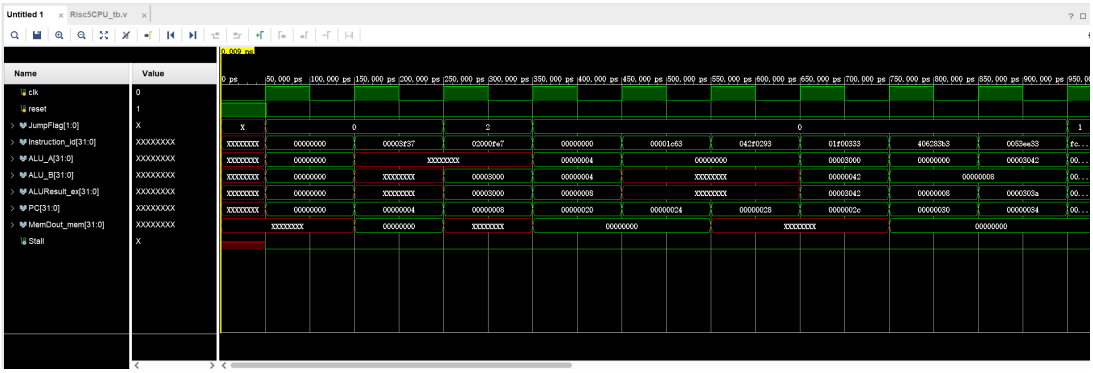


Figure 29: CPU 仿真结果放大图 1

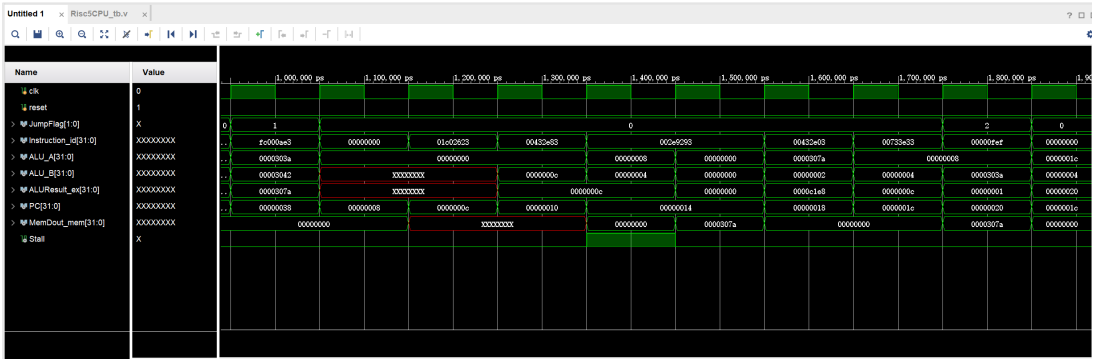


Figure 30: CPU 仿真结果放大图 2

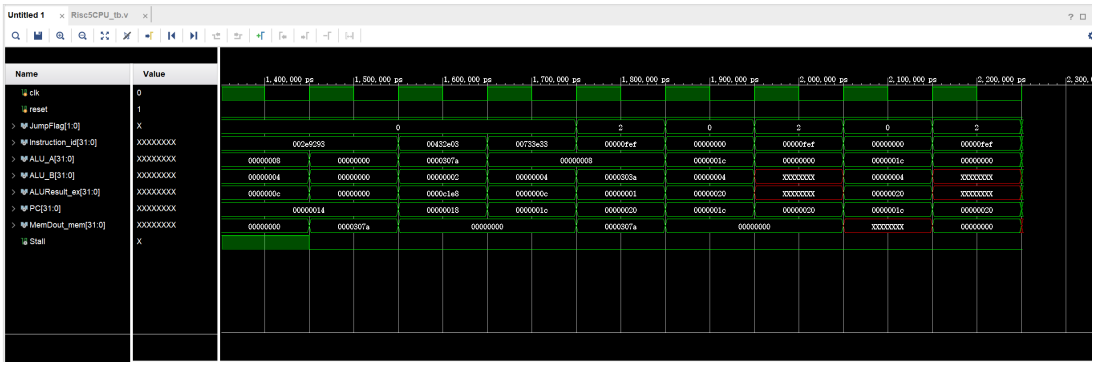


Figure 31: CPU 仿真结果放大图 3

对照表 30.11 测试程序运行结果，每个时钟周期中各个值的结果均正确，说明整体设计正确。

6 思考题

如下面两条指令所示，条件分支指令试图读取上一条指令的目标寄存器，此时无论插入气泡还是采用数据转发，都无法完全解决流水线冲突问题。为什么在大多数 CPU 架构中都不去专门解决这一问题？该问题通常应在什么层面中进行解决？

```
lw  X28, 04(X6)
beq X28, X29, Loop
```

上述指令中，条件分支指令 `beq` 试图立即读取上一条 `lw` 指令写回的目标寄存器 `X28`。由于 `lw` 指令的数据需在 MEM 阶段结束后才能获得，而分支判断通常在 ID 或 EX 阶段完成，因此出现了数据冒险。在该情况下，即使采用数据前推机制，也无法在分支判断阶段获得正确的数据；而仅通过插入气泡（流水线阻塞）虽然可以保证功能正确，但会显著增加流水线停顿次数，严重影响处理器性能。

因此，在大多数 CPU 架构中，通常不会针对该类特殊指令组合设计专门的硬件逻辑进行完全解决。所以该问题通常不在微处理器流水线硬件层面完全解决，而是更多地

在编译器层面或软件层面进行处理。

7 遇到的问题和解决方案

（1）在配置 IP 核的过程中出现了错误，检查设置之后发现是 data width 没改，导致功能失效，应该注意数据存储量是 64×32bit 的单端口 RAM，且输出采用 Non Registered 的组合输出。

（2）多模块编程的时候要注意变量名一致和信号位宽的匹配。在编写 ID 和 Decode 的过程中，一开始混淆了变量名，导致了控制信号传递错误和仿真结果的异常，反复检查之后才发现并改正了这些细节问题。

（3）由于对 Verilog 语言掌握不够熟练，我在编写 `always @(*)` 组合逻辑块时使用了非阻塞赋值（`<=`），导致综合失败。复习 verilog 之后发现，`always @(*)` 描述的是组合逻辑，其执行依赖输入信号变化，要求用阻塞赋值（=）确保信号实时更新、逻辑无歧义；而非阻塞赋值（<=）多用于时序逻辑（如 always @(posedge clk)），靠时钟边沿触发，信号赋值会延迟到周期结束。

8 设计总结

本次实验要求基于 RV32I 指令集设计并实现五级流水线 RISC-V 微处理器。处理器支持包括算术运算、逻辑运算、移位、分支跳转、数据传送在内的基本整数指令，并通过数据前推与流水线阻塞机制解决数据冒险问题，在 25MHz 以上的主频下稳定运行。

在实验过程中，我深入理解了流水线处理器各阶段（IF、ID、EX、MEM、WB）的分工与协作机制，掌握了数据相关、控制冒险的识别与处理方法，并通过模块化设计实现了指令译码、寄存器堆、ALU、存储器接口等关键部件。同时，通过使用 Vivado IP 核集成分布式 RAM，进一步熟悉了基于 FPGA 的存储系统设计流程。虽然在实验过程中碰到了很多问题，例如多模块设计的复杂、IP 核配置报错、verilog 语言语法使用不当等，但是通过仔细阅读实验指南、查阅资料、向老师请教等方法，还是逐一解决了各个问题。最终通过仿真与上板测试，我验证了处理器能够正确执行测试程序，各模块功能符合预期。

总体上说，本次实验不仅巩固了我在计算机组成与设计课程中学到的相关理论知识，也让我更可以熟练地编写 verilog 语言，在复杂数字系统设计、调试与验证中极大地提升了工程实践能力。